

Master Degree in Applied and Computational Mathematics
2025-2026

Master Thesis

“Data-Driven Discovery of Dynamics in Evolutionary Game Theory”

Nicolas Beuter

Advisor:

Miguel Angel Moscoso Castro

Madrid, 14.06.2026

SUMMARY

Simulations built from simple individual rules reproduce collective behavior, yet the macroscopic equations governing that behavior are rarely known in closed form. This work tests whether such equations can be recovered directly from simulated trajectories by data-driven discovery, using the Hawk-Dove game of evolutionary game theory as a test case. The variable of interest is the mean Hawk strategy, the average probability of playing the aggressive action in the population, which ranges from zero to one.

The method is the Sparse Identification of Nonlinear Dynamics (SINDy). The pipeline implemented in this work denoises each trajectory, fits a sparse combination of candidate functions by sequentially thresholded least squares, and selects the final model among the candidates with a corrected Akaike information criterion. Models are evaluated by their recovered structure, their fixed points and stability, and their predictive error on held-out trajectories. Under certain assumptions, the macroscopic limit of the game is the replicator equation, which is used as the reference for evaluation. Synthetic trajectories generated from it validate the pipeline under increasing noise. The core of the work then applies the same pipeline to data generated by a genetic-algorithm simulation, where the dynamics arise only from microscopic interaction and reproduction rules. The replicator equation is not imposed, and it is not known in advance whether these dynamics admit a governing equation at all. Two datasets are generated that differ only in the microscopic death rule.

The pipeline's robustness to noise is confirmed on the synthetic data, where it recovers the replicator equation for noise up to about fourteen percent of the state range. On the genetic-algorithm data the outcome depends on the microscopic death rule. When low-fitness individuals are more likely to die, the discovery selects a non-replicator polynomial structure that fits the data better than the replicator equation while sharing the same equilibrium. When every individual is equally likely to die, it instead recovers the replicator structure.

Interpretable macroscopic equations can therefore be discovered from the stochastic output of a genetic algorithm, and the discovery detects when a change in the microscopic rules alters the dynamics. The same pipeline could recover governing equations from other simulations of interacting individuals where none is known in closed form.

Keywords: data-driven discovery; sparse identification of nonlinear dynamics; evolutionary game theory; Hawk-Dove game; replicator equation; genetic algorithm.

CONTENTS

1. INTRODUCTION.	1
1.1. State of the Art	3
1.2. Structure of the Thesis	4
2. DATA-DRIVEN DISCOVERY OF DYNAMICS	5
2.1. The SINDy Framework	5
2.2. The STLSQ Optimizer.	9
2.3. Model Selection and Information Criteria	12
2.4. Evaluation Metrics	14
3. THE HAWK-DOVE GAME	17
3.1. The Hawk-Dove Game and the Replicator Equation	17
3.2. Simulating Evolutionary Dynamics via Genetic Algorithms.	21
4. METHODOLOGY	24
4.1. Data Generation and Preprocessing	24
4.2. The Discovery Pipeline	30
4.3. Evaluation Protocol	33
5. RESULTS	35
5.1. RQ1: Pipeline Design and Robustness to Noise (Synthetic Data)	35
5.2. RQ2: The Literature Baseline (Inverse Death Rule)	39
5.3. RQ3: Microscopic Sensitivity (Uniform Death Rule).	42
6. DISCUSSION	45
6.1. Interpretation of the Results	45
6.2. Limitations and Difficulties	46
6.3. Future Research	47
7. CONCLUSION	48
BIBLIOGRAPHY.	49
A. SAVITZKY-GOLAY FILTER	
B. PSEUDOCODE FOR THE GENETIC ALGORITHM	

C. MODEL EVALUATION	
C.1. Synthetic Data	
C.2. GA Data (Inverse Death)	
C.3. GA Data (Uniform Death)	

LIST OF FIGURES

2.1	Schematic of Pareto front and AIC score comparison	13
4.1	Ensemble averaging of genetic algorithm trajectories	26
4.2	Ensemble-averaged genetic algorithm fitting trajectories	28
5.1	Synthetic data - trajectory predictions at $\eta = 8\%$	37
5.2	Synthetic data - sensitivity of recovery quality	38
5.3	GA data (inverse death) - trajectory predictions	40
5.4	GA data (inverse death) - sensitivity of recovery quality	41
5.5	GA data (uniform death) - trajectory predictions	43
5.6	GA data (uniform death) - sensitivity of recovery quality	44

LIST OF TABLES

3.1	Expected payoff matrix of the Hawk-Dove game	18
3.2	Realized payoff $\delta w_{i,k}^t$ of an individual in a single encounter.	22
4.1	Parameters for the genetic algorithm experiment.	29
4.2	Parameters of the SINDy discovery pipeline.	32
5.1	Synthetic data - model selection	36
5.2	GA data (inverse death) - model selection	39
5.3	GA data (uniform death) - model selection	42
C.1	Synthetic data - best model evaluation	
C.2	GA data (inverse death) - best model evaluation	
C.3	GA data (uniform death) - best model evaluation	

1. INTRODUCTION

The main objective of this work is to test whether interpretable macroscopic governing equations can be recovered directly from the trajectories of a genetic algorithm by data-driven discovery, and how such a pipeline must be designed to do so reliably. It is also investigated whether the discovered equation changes when the microscopic rules of the simulation are changed.

Understanding how such macroscopic laws emerge from microscopic interactions is a fundamental challenge in many fields. It appears in biology, ecology, economics, and social science. Agent-based models are a common tool for studying such systems. They encode simple rules at the level of individuals and let collective dynamics emerge from their repeated interactions.

Evolutionary game theory is one such field. A population evolves through repeated interaction and reproduction in discrete time steps. At each step an emergent quantity can be measured, for example the average strategy played in the population. The sequence of these measurements over one run gives a trajectory, which is the output of a simulation of microscopic interactions rather than the solution of a known equation.

Different initial conditions in such simulations can lead to very different outcomes. A strategy may spread through the whole population, die out, or settle at a stable equilibrium. The trajectories show that such phenomena occur, but they give little insight into why. They do not provide a generalizable or interpretable description of the dynamics. For that one would like a macroscopic governing equation, a differential equation that gives the rate of change of the state as a function of the state itself. For most complex agent-based systems such an equation cannot be derived analytically from the microscopic rules. The trajectories of emergent quantities, however, can be simulated and recorded. Recovering the governing equation from measured trajectories is the *inverse problem*. Data-driven methods address it by discovering the equation directly from data. The Sparse Identification of Nonlinear Dynamics (SINDy) algorithm was designed for exactly this situation [1]. It expresses the rate of change of each variable as a linear combination of candidate functions from a predefined library, where only a few terms are active. This *sparsity* assumption is well motivated. Most physical laws, from Newton's equations to population dynamics, are concise and involve only a few terms. Sparse models are also less likely to overfit, so they capture the true structure of the system rather than the noise. A further advantage of SINDy is that its output is *interpretable*. Black-box models such as neural networks can also predict trajectories accurately, but they give little insight into the underlying dynamics. SINDy instead returns a set of explicit differential equations. From these equations one can identify the fixed points and analyze their stability, determine which terms drive the dynamics, extract the underlying parameters, and understand how the system responds to changes in initial conditions. The precise

mathematical formulation is given in Chapter 2.

So far SINDy has mostly been applied to physical and biological systems, and only a few works have used it on the stochastic output of agent-based simulations (Section 1.1).

This work applies model discovery to the Hawk-Dove game [2], the archetypal example of evolutionary game theory. It describes a large population in which individuals repeatedly compete over a shared resource by playing one of two actions. A Hawk fights for the resource and can win it, but two Hawks risk injury. A Dove never fights and avoids injury, but yields the resource to a Hawk. Each individual plays Hawk with some probability p and Dove otherwise. Individuals that earn higher payoffs reproduce more, and offspring inherit the strategy of their parent. The composition of the population therefore changes over time, and with it the average probability of playing Hawk. This population average, the mean Hawk strategy $\bar{p}$, is the macroscopic quantity of interest. The Hawk-Dove game is well suited as a test case for several reasons. Its macroscopic dynamics are known in closed form and provide a benchmark to validate against. A published genetic algorithm of the game also exists, generating data directly from microscopic rules rather than from a governing equation. Finally, the dynamics converge to a stable interior equilibrium rather than to a boundary, which makes them interesting to recover.

Under certain assumptions, the dynamics of $\bar{p}$ are governed by the replicator equation. It is a one-dimensional differential equation and is known in closed form. The discovery pipeline is first validated on synthetic data generated by integrating this equation, and its robustness to noise is tested. This motivates the first research question, which concerns the design of the method itself.

RQ1 How must a data-driven recovery pipeline be designed to reliably extract underlying dynamical equations from noisy trajectory data?

Validating on synthetic data confirms that the pipeline works, but this data is generated from the replicator equation itself. Recovering that same equation is therefore not a real test, since the answer is already built into the data. To test the method on data that does not come from a known equation, a genetic algorithm is used to simulate the Hawk-Dove game. This work uses the genetic algorithm of Laruelle et al. [3]. A finite population of individuals plays stochastic encounters and reproduces through microscopic birth and death rules. The mean Hawk strategy $\bar{p}$ is stored at each generation, which yields one stochastic trajectory per run. Trajectories from different initial conditions are then passed to the discovery pipeline.

This setup avoids an inverse crime, which would mean generating data from an equation and then recovering that same equation. Here the data come only from the microscopic interactions of the genetic algorithm, and the discovery process never imposes the replicator equation, so recovering a governing equation here tests the discovery process on a system whose dynamics are not known in advance. Two datasets are produced in this way, differing only in the death rule, the rule that decides which individuals are removed

after each generation. The first uses the inverse-proportional death rule of the reference paper, under which low-fitness individuals are more likely to die. The second uses a uniform death rule, under which every individual is equally likely to be removed. Comparing the equations discovered from the two datasets tests whether the method is sensitive to this single microscopic change. These two experiments motivate the remaining two research questions.

RQ2 What differential equation best describes the macroscopic dynamics of the genetic algorithm proposed by Laruelle et al. [3]?

RQ3 Does the discovery algorithm detect another governing equation if the microscopic death rule in the genetic algorithm is changed?

1.1. State of the Art

This section reviews how methods for learning governing equations from data have developed, and where the method used in this work fits in.

Early methods used symbolic regression. Bongard and Lipson [4] and Schmidt and Lipson [5] searched through possible mathematical expressions for the differential equation that best fits the data with few terms. This recovered physical laws directly from data, but the search is slow, scales poorly, and can overfit. SINDy [1] avoids this search by fixing a library of candidate terms and using sparse regression to keep only the few that are active.

Since then, each part of the method has been improved. The original sparse fit by sequential thresholded least squares (STLSQ) now has several alternative optimizers, including an ℓ_1 -based formulation [6], a unified framework that also enforces constraints [7], and SINDy-PI for implicit and rational dynamics [8]. Noise is the main difficulty, because estimating the required time derivative of the state amplifies it. The weak formulation avoids the pointwise derivative by integrating against test functions [9], while ensemble methods fit many models on resampled data and average them [10]. Once several candidate models are produced, they must be compared, which Mangan et al. [11] do with information criteria that weigh the fit against the number of terms. These methods are collected in the open-source package PySINDy [12], [13]. This work uses its standard optimizer, with simple smoothing for the noise and the corrected Akaike information criterion for the selection.

The method has also been extended well beyond this setting, to partial differential equations [14] or to regressions constrained by known physics [15]. These extensions are not needed here, where the state is already the macroscopic quantity and the dynamics are a one-dimensional ordinary differential equation. SINDy has been applied across many fields, for example to biological networks [16] and to recover models of infectious disease spread from case data [17].

SINDy has also been used on the stochastic output of agent-based simulations, the setting of this thesis. Nardini et al. [18] do this for agent-based models of cell biology and epidemiology, and Messenger and Bortz recover the mean-field equation of an interacting particle system from simulated particle positions [19]. These works show that a macroscopic equation can be recovered from a microscopic stochastic simulation. This thesis follows the same idea for evolutionary game theory, where the data come from a genetic algorithm of a population playing repeated games, and it also tests whether the recovered equation changes when the microscopic rules change.

1.2. Structure of the Thesis

The remainder of this work is structured as follows. Chapter 2 introduces the SINDy framework, covering the sparse regression method, the model selection criterion, and the evaluation metrics used throughout. Chapter 3 presents the Hawk-Dove game, derives the replicator equation, and describes the genetic algorithm used to simulate its dynamics. Chapter 4 details the experimental setup, including data generation, preprocessing, and the identification pipeline. Chapter 5 reports the results for the synthetic data and the two genetic algorithm datasets. Chapter 6 discusses the findings and answers the research questions. Chapter 7 concludes the work.

2. DATA-DRIVEN DISCOVERY OF DYNAMICS

This chapter develops the mathematical framework for data-driven discovery of governing equations, following the SINDy approach of Brunton et al. [1]. Section 2.1 sets up the framework. It organizes the trajectory data, their numerical derivatives, and a library of candidate functions, and turns the discovery of the governing equation into a sparse regression problem. Section 2.2 presents the Sequentially Thresholded Least Squares (STLSQ) algorithm, which solves this problem. Section 2.3 introduces the corrected Akaike Information Criterion (AIC_c) to select the best model from the resulting candidates. Finally, Section 2.4 defines the metrics used to evaluate the identified model.

2.1. The SINDy Framework

The aim of SINDy is to recover the governing equation of a dynamical system directly from measured data. The system is assumed to obey an autonomous ordinary differential equation

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)), \quad (2.1)$$

where the function $\mathbf{f}$ describes the dynamics and is unknown¹. SINDy assumes that $\mathbf{f}$ is built from only a few active terms, so most candidate terms do not contribute. The goal is to identify which terms are active and what their coefficients are, using only samples of the state and estimates of its time derivative.

The method proceeds in four stages. The measured states are first organized into a data matrix (Section 2.1.1). The time derivatives are then obtained by numerical differentiation (Section 2.1.2). Next, a library of candidate functions is constructed (Section 2.1.3). Finally, the identification is formulated as a sparse optimization problem (Section 2.1.4).

2.1.1. Data Acquisition

For an n -dimensional system, the state at a given time t is described by the column vector $\mathbf{x}(t) = [x_1(t), \dots, x_n(t)]^T$. Its n entries are the variables that describe the system. A trajectory is the evolution of this state from one initial condition. The state is recorded at discrete, ordered time points $t_1 < t_2 < \dots$, which gives an ordered sequence of states $\mathbf{x}(t_1), \mathbf{x}(t_2), \dots$. Each initial condition leads to a different trajectory.

Data-driven discovery needs trajectory data that is rich enough to reveal the dynamics. A single trajectory follows only one path through the state space. Several trajectories with different initial conditions cover a wider region, so they make the discovery more reliable.

¹Throughout this work, bold letters denote vectors and matrices.

Suppose r trajectories are recorded. Trajectory j starts from its own initial condition and is sampled at m_j time points, giving the states $\mathbf{x}^{(j)}(t_1), \dots, \mathbf{x}^{(j)}(t_{m_j})$. The total number of snapshots across all trajectories is

$$m = \sum_{j=1}^r m_j. \quad (2.2)$$

These m snapshots are combined into a single dataset. Each recorded state becomes one row of a matrix. The rows are stacked trajectory by trajectory, first the samples of trajectory 1, then those of trajectory 2, and so on. This gives the snapshot matrix

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^T(t_1) \\ \mathbf{x}^T(t_2) \\ \vdots \\ \mathbf{x}^T(t_m) \end{bmatrix} = \begin{bmatrix} x_1(t_1) & x_2(t_1) & \cdots & x_n(t_1) \\ x_1(t_2) & x_2(t_2) & \cdots & x_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(t_m) & x_2(t_m) & \cdots & x_n(t_m) \end{bmatrix} \in \mathbb{R}^{m \times n}, \quad (2.3)$$

where each row holds the state at one time point of one trajectory.

2.1.2. Numerical Differentiation

The identification problem needs the time derivative of the state at each recorded point. These derivatives are not measured directly and must be estimated from the sampled states. Real trajectory data are noisy, so instead of the true state $\mathbf{x}_{true}(t)$ one only has a corrupted version

$$\tilde{\mathbf{x}}(t_i) = \mathbf{x}_{true}(t_i) + \boldsymbol{\varepsilon}(t_i), \quad (2.4)$$

where $\boldsymbol{\varepsilon}(t_i) \in \mathbb{R}^n$ is the noise vector at time t_i . Taking finite differences between such noisy points amplifies the noise and makes the estimated derivatives unreliable.

The Savitzky-Golay filter [20] estimates the derivative in a more robust way. It slides a window of $W = 2w + 1$ consecutive measurements along the trajectory and fits a low-degree polynomial to the points in each window by least squares. The derivative of this polynomial at the center of the window is taken as the estimated time derivative there. PySINDy recommends the filter as one of its methods for estimating derivatives from noisy data [13], and it is the approach used in this work. The full procedure, including how the same filter can also be used to smooth the state values, is given in Appendix A.

The filter is applied to each trajectory separately, since estimating a derivative across the boundary between two trajectories would be meaningless. For each trajectory it returns a time derivative at every recorded time point. These derivatives are stacked, in the same order as the states in Section 2.1.1, into the derivative matrix

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{\mathbf{x}}^T(t_1) \\ \dot{\mathbf{x}}^T(t_2) \\ \vdots \\ \dot{\mathbf{x}}^T(t_m) \end{bmatrix} = \begin{bmatrix} \dot{x}_1(t_1) & \cdots & \dot{x}_n(t_1) \\ \dot{x}_1(t_2) & \cdots & \dot{x}_n(t_2) \\ \vdots & \ddots & \vdots \\ \dot{x}_1(t_m) & \cdots & \dot{x}_n(t_m) \end{bmatrix} \in \mathbb{R}^{m \times n}. \quad (2.5)$$

Because both matrices use the same ordering, row i of $\dot{\mathbf{X}}$ is the derivative of the state in row i of $\mathbf{X}$, for $i = 1, \dots, m$.

2.1.3. Construction of the Candidate-function Library

The states and their derivatives are now organized in $\mathbf{X}$ and $\dot{\mathbf{X}}$. To find the function $\mathbf{f}$ in Eq. (2.1), one first builds a library of candidate functions, from which SINDy later selects the active terms of $\mathbf{f}$.

The candidate functions are the terms that could appear in $\mathbf{f}$. For a generic state $\mathbf{x}$ they are collected in the symbolic row library vector

$$\Theta_{\text{symbolic}}(\mathbf{x}) = [1 \quad x_1 \quad \cdots \quad x_n \quad x_1^2 \quad x_1 x_2 \quad \cdots \quad x_n^2 \quad \cdots], \quad (2.6)$$

which here contains a constant, the linear terms x_i , the quadratic terms x_i^2 and $x_i x_j$, and higher-order monomials. The library is a modeling choice and is not restricted to polynomials. Depending on the system, other nonlinear terms such as trigonometric functions $\sin(\mathbf{x})$, $\cos(\mathbf{x})$ or exponentials $\exp(\mathbf{x})$ can be included.

Each candidate function is evaluated at every measured state. Stacking these evaluations over all m snapshots gives the data library matrix

$$\Theta(\mathbf{X}) = \begin{bmatrix} | & | & | & | & \cdots \\ \mathbf{1} & \mathbf{X}^{(P1)} & \mathbf{X}^{(P2)} & \mathbf{X}^{(P3)} & \cdots \\ | & | & | & | & \end{bmatrix} \in \mathbb{R}^{m \times l}, \quad (2.7)$$

where each column holds one candidate function evaluated along all m snapshots, and the blocks group functions of the same type. The first block is a column of ones for the constant, $\mathbf{X}^{(P1)}$ holds the linear terms, $\mathbf{X}^{(P2)}$ the second-degree monomials, and so on. For example, $\mathbf{X}^{(P2)}$ contains all quadratic terms x_i^2 and all cross-terms $x_i x_j$ with $i < j$ and $i, j \in \{1, \dots, n\}$,

$$\mathbf{X}^{(P2)} = \begin{bmatrix} x_1^2(t_1) & x_1(t_1)x_2(t_1) & \cdots & x_2^2(t_1) & x_2(t_1)x_3(t_1) & \cdots & x_n^2(t_1) \\ x_1^2(t_2) & x_1(t_2)x_2(t_2) & \cdots & x_2^2(t_2) & x_2(t_2)x_3(t_2) & \cdots & x_n^2(t_2) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_1^2(t_m) & x_1(t_m)x_2(t_m) & \cdots & x_2^2(t_m) & x_2(t_m)x_3(t_m) & \cdots & x_n^2(t_m) \end{bmatrix} \in \mathbb{R}^{m \times \frac{n(n+1)}{2}}. \quad (2.8)$$

The choice of the library matters. If it does not contain the true governing terms, the true equation cannot be recovered. Even then, the algorithm may select other functions from the library that approximate the dynamics well over the range of states explored by the data. The resulting model can be predictive on the available data while remaining structurally different from the true underlying system.

2.1.4. Sparse Identification of the Governing Equations

SINDy assumes that each component of $\mathbf{f}$ in Eq. (2.1) can be represented as a linear combination of the candidate functions in the library, with only a few nonzero coefficients.

For a state $\mathbf{x}$, the k -th component of the dynamics is then written as

$$\dot{x}_k = f_k(\mathbf{x}) \approx \Theta_{\text{symbolic}}(\mathbf{x}) \xi_k, \quad (2.9)$$

where $\Theta_{\text{symbolic}}(\mathbf{x})$ is the symbolic library vector from Eq. (2.6) and $\xi_k \in \mathbb{R}^l$ holds the coefficients of the candidate functions for component k . This representation is only the starting point. The coefficients ξ_k are unknown, and determining them is the identification problem, made precise below. Collecting the coefficient vectors of all n components gives the coefficient matrix

$$\Xi = \begin{bmatrix} | & | & & | \\ \xi_1 & \xi_2 & \cdots & \xi_n \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{l \times n}. \quad (2.10)$$

The objective of expressing $\mathbf{f}$ with only a few candidate functions can now be expressed formally. Each coefficient vector ξ_k should be sparse, so most of its entries are zero,

$$\|\xi_k\|_0 = |\{j \in \{1, \dots, l\} : \xi_{k,j} \neq 0\}| \ll l, \quad (2.11)$$

which means only a few candidate functions are active for each component.

The model in Eq. (2.9) should hold at every measured state. Evaluating it at all m snapshots replaces the symbolic vector $\Theta_{\text{symbolic}}(\mathbf{x})$ by the data library matrix $\Theta(\mathbf{X})$ of Eq. (2.7). Together with the derivative matrix $\dot{\mathbf{X}}$, this gives the overdetermined linear system

$$\dot{\mathbf{X}} \approx \Theta(\mathbf{X}) \Xi, \quad (2.12)$$

where the approximation reflects that the model is fitted to the data rather than required to hold exactly. Since the columns of Ξ decouple, this splits into n independent problems, one for each state dimension k , in which the k -th column $\dot{\mathbf{X}}_k$ of $\dot{\mathbf{X}}$ is fitted by $\Theta(\mathbf{X}) \xi_k$. The identification is therefore stated below for a single coefficient vector ξ_k .

The identification problem is to find, for each component, the sparsest coefficient vector that fits the system. In the ideal case where the model holds exactly, this is the canonical sparse-recovery problem of finding the sparsest solution of a linear system [21], [22],

$$\xi_k = \arg \min_{\xi \in \mathbb{R}^l} \|\xi\|_0 \quad \text{subject to} \quad \Theta(\mathbf{X}) \xi = \dot{\mathbf{X}}_k, \quad (2.13)$$

which selects the fewest candidate functions consistent with the data, where $\|\xi_k\|_0$ counts the active terms as in Eq. (2.11).

Problem (2.13) is combinatorial. The ℓ_0 count is not a norm and the objective is nonconvex, so finding its minimizer requires in principle a search over all 2^l possible supports, which is NP-hard [23].

Two features of the present setting also prevent the equality constraint from being imposed. The system is overdetermined, since $\Theta(\mathbf{X})$ has many more rows than columns

($m \gg l$), and the derivative data $\dot{\mathbf{X}}_k$ is noisy, so in general no ξ satisfies $\Theta(\mathbf{X})\xi = \dot{\mathbf{X}}_k$ exactly, which is why Eq. (2.12) holds only approximately. The equality constraint is therefore relaxed into the objective as a least-squares residual term, giving the penalized problem

$$\xi_k = \arg \min_{\xi \in \mathbb{R}^l} \|\dot{\mathbf{X}}_k - \Theta(\mathbf{X})\xi\|_2^2 + \lambda^2 \|\xi\|_0, \quad (2.14)$$

where the first term measures how well the model reproduces the derivative data, the second counts the active terms through the ℓ_0 count weighted by λ^2 . The parameter $\lambda > 0$ sets the balance between accuracy and sparsity. Unlike classical sparse recovery, where the system is underdetermined and sparsity can make the solution unique [22], here the sparsity term acts as a criterion that favors a simple model among the many that could fit the overdetermined data.

Equation (2.14) is the central problem that SINDy solves, and it is a sparse regression problem. The ℓ_0 penalty makes it nonconvex and generally intractable to solve exactly. Its solution is therefore approximated, either by replacing the ℓ_0 penalty with the convex ℓ_1 norm [24] or by greedy thresholding. This work uses the Sequentially Thresholded Least Squares (STLSQ) algorithm, a greedy scheme presented in Section 2.2. If the identification is successful, a good approximation of the dynamics has been found, as motivated in Eq. (2.9).

2.2. The STLSQ Optimizer

The identification problem in Eq. (2.14) is solved approximately by the Sequentially Thresholded Least Squares (STLSQ) algorithm of Brunton et al. [1], a greedy iterative scheme. The algorithm is described first, followed by two supporting steps, the normalization of the library applied before the regression and a final unbiased refit that corrects the coefficient estimates.

2.2.1. The STLSQ Algorithm

As noted in Section 2.1.4, the identification problem decouples across state dimensions, so STLSQ is described for a single component k . It approximates the solution of Eq. (2.14) for the coefficient vector ξ_k , using the derivative column $\dot{\mathbf{x}}_k$ and the library. So that the threshold introduced below acts comparably across candidate functions, the algorithm operates on the column-normalized library $\tilde{\Theta}$, in which every column is scaled to unit norm. The reason for this and its construction are explained afterwards in Section 2.2.2.

Solving Eq. (2.14) exactly would mean trying every subset of candidate functions and keeping the best sparse one. This combinatorial ℓ_0 minimization is NP-hard and infeasible for a large library. STLSQ avoids this search and builds the sparse model by pruning. It removes the candidate functions whose coefficients fall below a threshold and refits the model on the ones that remain, repeating until only the few important terms are left.

The algorithm starts from the least-squares fit that uses all candidate functions,

$$\xi_k^{(0)} = \bar{\Theta}^\dagger \dot{\mathbf{x}}_k, \quad (2.15)$$

where $\bar{\Theta}^\dagger$ is the Moore-Penrose pseudoinverse². This is the best fit obtainable from the full library when sparsity is not required, so it is generally dense. The size of each coefficient already indicates how much its term contributes. Terms that drive the dynamics receive large coefficients, while terms that do not receive small ones, and it is this ranking that the threshold acts on.

After this initialization the algorithm alternates two steps. The first is hard thresholding. All coefficients with magnitude below the threshold λ are set to zero, which removes the least-contributing terms and leaves the active support

$$\mathcal{S}_k^{(i)} = \left\{ j \in \{1, \dots, l\} : |\xi_{k,j}^{(i)}| \geq \lambda \right\}. \quad (2.16)$$

The second is a least-squares refit restricted to this support,

$$\xi_k^{(i+1)}[\mathcal{S}_k^{(i)}] = \bar{\Theta}_{\mathcal{S}_k^{(i)}}^\dagger \dot{\mathbf{x}}_k, \quad \xi_{k,j}^{(i+1)} = 0 \text{ for } j \notin \mathcal{S}_k^{(i)}. \quad (2.17)$$

The refit lets the surviving coefficients readjust to account for the removed terms, so the fit stays close to the data while the model becomes sparser.

The two steps repeat until the support no longer changes. Each pass prunes the library further, down to the few terms that drive the dynamics.

The threshold λ is the sparsity parameter of the algorithm, and it is directly tied to the ℓ_0 penalty in Eq. (2.14). Hard-thresholding is the exact solution of the ℓ_0 problem for a single coefficient, which is kept only when keeping it reduces the squared residual by more than the penalty. Because the residual is measured in squared units while the threshold acts on the coefficient magnitude, a threshold λ corresponds to the penalty weight λ^2 in Eq. (2.14). In this sense λ is the sparsity parameter of the identification problem itself, and a larger λ removes more terms and yields a sparser model.

A term that is thresholded out is never brought back, the active support can only shrink across iterations. STLSQ is therefore a greedy scheme. It does not minimize Eq. (2.14) exactly, but each iteration decreases the objective, so the algorithm converges to a local minimizer [1, SI Appendix, Sec. 3.1].

The least-squares refit in Eq. (2.17) can become unstable when the active columns are nearly collinear, since the submatrix $\bar{\Theta}_{\mathcal{S}_k^{(i)}}$ is then ill-conditioned and the solution has high variance. To stabilize it, the refit is replaced by a ridge-regularized least-squares problem [25][Sec. 3.4.1],

$$\xi_k^{(i+1)} = \arg \min_{\xi} \left\| \dot{\mathbf{x}}_k - \bar{\Theta}_{\mathcal{S}_k^{(i)}} \xi \right\|_2^2 + \alpha \|\xi\|_2^2, \quad (2.18)$$

²The Moore-Penrose pseudoinverse of a matrix $\mathbf{A} \in \mathbb{R}^{m \times l}$, denoted $\mathbf{A}^\dagger$, generalizes the matrix inverse to non-square and rank-deficient matrices. For a full column-rank matrix with $m \geq l$, it reduces to $\mathbf{A}^\dagger = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$, and the solution $\hat{\mathbf{x}} = \mathbf{A}^\dagger \mathbf{b}$ minimizes $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$ over all $\mathbf{x} \in \mathbb{R}^l$.

with regularization parameter $\alpha \geq 0$. This ridge extension follows the PySINDy implementation [12] used in this work. The added term $\alpha \mathbf{I}$ makes the system matrix $\bar{\Theta}_{S_k}^T \bar{\Theta}_{S_k} + \alpha \mathbf{I}$ strictly positive definite and therefore invertible, which gives the closed-form refit

$$\xi_k^{(i+1)}[S_k^{(i)}] = \left(\bar{\Theta}_{S_k^{(i)}}^T \bar{\Theta}_{S_k^{(i)}} + \alpha \mathbf{I} \right)^{-1} \bar{\Theta}_{S_k^{(i)}}^T \dot{\mathbf{x}}_k. \quad (2.19)$$

For $\alpha = 0$ this reduces to the ordinary least-squares refit of Eq. (2.17), recovering the classical STLSQ.

The complete procedure is summarized in Algorithm 1.

Algorithm 1 Sequentially Thresholded Least Squares (STLSQ) for component k

Input: $\bar{\Theta} \in \mathbb{R}^{m \times l}$, $\dot{\mathbf{x}}_k \in \mathbb{R}^m$, threshold $\lambda > 0$, regularization $\alpha \geq 0$, max iterations R

Output: Sparse coefficient vector $\xi_k \in \mathbb{R}^l$

- 1: **Initialize:** $\xi_k^{(0)} \leftarrow \bar{\Theta}^\dagger \dot{\mathbf{x}}_k$, $i \leftarrow 0$
 - 2: **repeat**
 - 3: $S_k^{(i)} \leftarrow \{j \in \{1, \dots, l\} : |\xi_{k,j}^{(i)}| \geq \lambda\}$
 - 4: $\xi_k^{(i+1)}[S_k^{(i)}] \leftarrow (\bar{\Theta}_{S_k^{(i)}}^T \bar{\Theta}_{S_k^{(i)}} + \alpha \mathbf{I})^{-1} \bar{\Theta}_{S_k^{(i)}}^T \dot{\mathbf{x}}_k$
 - 5: $\xi_{k,j}^{(i+1)} \leftarrow 0$ **for all** $j \notin S_k^{(i)}$
 - 6: $i \leftarrow i + 1$
 - 7: **until** $S_k^{(i)} = S_k^{(i-1)}$ **or** $i = R$
 - 8: **return** $\xi_k^{(i)}$
-

The iteration stops when the active support no longer changes between two steps, or after a maximum of R iterations, which guarantees termination. Applying the algorithm independently to each component $k = 1, \dots, n$ gives the full coefficient matrix $\Xi = [\xi_1 \mid \dots \mid \xi_n] \in \mathbb{R}^{l \times n}$

2.2.2. Column Normalization of the Library

The STLSQ algorithm removes a term when its coefficient falls below the threshold λ . The size of a fitted coefficient depends on the scale of the corresponding library column, so a single threshold applied to the raw library would treat the candidate functions unequally. To avoid this, each column of the data library matrix $\Theta(\mathbf{X})$ (Eq. (2.7)) is scaled to unit Euclidean norm before the regression. Let $\theta_j \in \mathbb{R}^m$ denote the j -th column, with norm $d_j = \|\theta_j\|_2$. The normalized library is

$$\bar{\Theta} = \Theta(\mathbf{X}) \mathbf{D}^{-1}, \quad \mathbf{D} = \text{diag}(d_1, \dots, d_l), \quad (2.20)$$

so that every column of $\bar{\Theta}$ has unit norm.

The need for this step is clearest with an example. When the state values lie in $(0, 1)$, raising them to a high power gives entries much smaller than one, so a column such as x_k^4 has small entries and a small Euclidean norm. The constant column $\mathbf{1}$, by contrast, has norm $\sqrt{m}$. Without normalization a fixed λ would keep high-degree terms, whose

coefficients are inflated by the small column scale, and remove low-degree terms, whose coefficients are deflated by the large column scale, regardless of their actual contribution to the dynamics. Scaling all columns to unit norm removes this dependence and lets λ act uniformly across the candidate functions [1, SI Appendix, Sec. 3].

Because the regression is carried out on $\bar{\mathbf{\Theta}}$, the coefficients it returns are on the normalized scale. Once STLSQ has converged and returned the normalized coefficients $\bar{\mathbf{\Xi}}$, they are mapped back to the original scale by

$$\hat{\mathbf{\Xi}} = \mathbf{D}^{-1} \bar{\mathbf{\Xi}}. \quad (2.21)$$

2.2.3. Bias Correction via Refitting

When $\alpha > 0$, every refit inside the loop uses ridge regression, which shrinks all coefficient estimates toward zero. The selected support is then still correct, but the magnitudes of the surviving coefficients are underestimated. To remove this bias, a final ordinary least-squares refit is performed on the converged support $\mathcal{S}_k$, without regularization,

$$\bar{\mathbf{\xi}}_k = \bar{\mathbf{\Theta}}_{\mathcal{S}_k}^\dagger \mathbf{\dot{x}}_k, \quad (2.22)$$

where $(\cdot)^\dagger$ denotes the Moore-Penrose pseudoinverse. This refit keeps the identified support unchanged and only recalibrates the coefficient magnitudes to their unbiased least-squares values on that support [12, Sec. 5.4]. The resulting coefficients are then mapped back to the original scale by Eq. (2.21).

2.3. Model Selection and Information Criteria

The STLSQ algorithm depends on two hyperparameters, the sparsity threshold λ and the ridge parameter α , and there is no obvious choice for them in advance. Different values lead to different models. So running the algorithm over a range of them produces a whole set of candidate models rather than a single one. Each candidate is a valid sparse equation, but they differ in which terms are active and in how well they reproduce the data. A small threshold keeps many terms and fits the data closely but risks overfitting. A large threshold keeps few terms and may miss part of the dynamics. Model selection is the task of choosing, from this set of candidates, the one that best describes the dynamics. It balances predictive accuracy against model complexity. The rest of this section develops a criterion that makes this choice objective.

The original SINDy implementation suggests using cross-validation to identify a sparse solution on the Pareto frontier ³[1, SI Appendix, Sec. 3.2]. In this context, cross-validation involves evaluating various hyperparameter combinations by simulating the

³The Pareto frontier, or Pareto front, is the set of solutions for which no objective can be improved without worsening another. Here the two competing objectives are predictive accuracy and model complexity: a solution is on the Pareto front if no other candidate achieves both lower error and fewer active terms simultaneously.

resulting models and measuring their predictive performance on held-out validation data. As illustrated in Fig. 2.1, the predictive error is plotted against model complexity, defined by the number of active terms. Beyond a certain point, adding further terms to the equations yields diminishing returns in predictive accuracy and increases the risk of overfitting. According to the principle of sparsity, the best model is generally located in the "elbow" of this Pareto front (gray box in Figure 2.1), where low error and minimal complexity are optimally balanced.

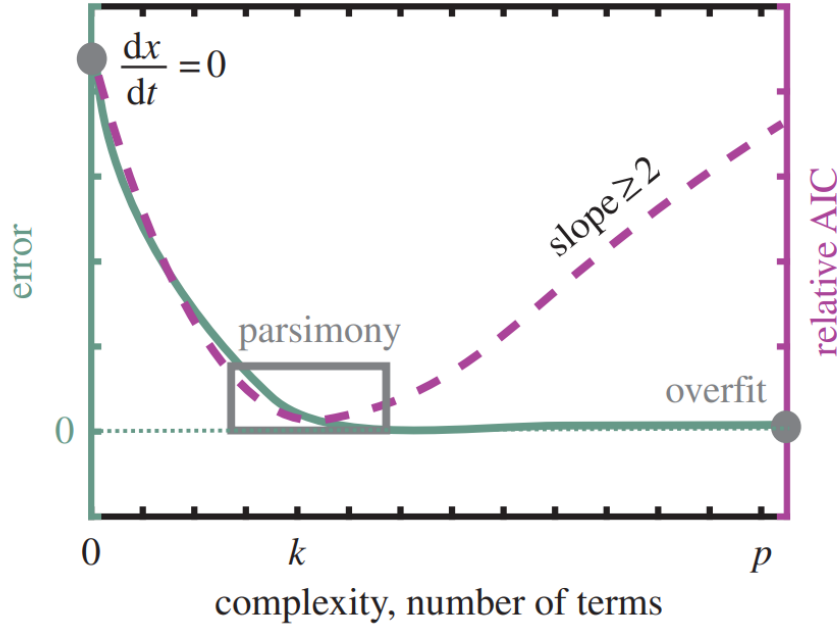


Fig. 2.1. Schematic comparison of model error (solid green) and the relative AIC score (dashed magenta) relative to model complexity. The gray box highlights the region of sparse models that optimally balance predictive error and model size ("elbow" of this Pareto front). While the error typically decreases as terms are added, the AIC score penalizes overfitting, indicated by the right-most gray dot, and identifies the most informative model at the curve's minimum. Taken from [11].

However, if this elbow is not clearly defined, a cluster of candidate models with similar performance may emerge, making an objective decision difficult. To resolve this problem, Mangan et al. [11] propose integrating tools from Information Theory to rank candidate models near the Pareto frontier. This approach uses scores that account for two fundamental properties: the goodness-of-fit on held-out validation data and a penalty for model complexity to prevent overfitting. The Akaike Information Criterion (AIC) provides such a score.

Each candidate model is forward-simulated (see next section for details) from the initial conditions of the held-out test trajectories. The goodness-of-fit of each trajectory i is quantified by its mean squared error:

$$\text{MSE}_i = \frac{1}{T} \sum_{\tau=1}^T \left\| \mathbf{x}_{\text{test},i}(\tau) - \hat{\mathbf{x}}_i(\tau; \hat{\xi}) \right\|_2^2, \quad (2.23)$$

where T is the number of time steps per test trajectory, $\mathbf{x}_{\text{test},i}(\tau)$ is the observed state at time step τ of the i -th test trajectory, and $\hat{\mathbf{x}}_i(\tau; \hat{\xi})$ is the corresponding forward-simulated state produced by integrating the candidate model with coefficient vector $\hat{\xi}$. The division by T normalises the error to a per-time-step scale, preventing longer trajectories from accumulating larger errors simply due to their length, and keeping the values interpretable as average squared deviations per step. The mean over all $\tilde{m}$ test trajectories is:

$$\overline{\text{MSE}} = \frac{1}{\tilde{m}} \sum_{i=1}^{\tilde{m}} \text{MSE}_i. \quad (2.24)$$

Let k denote the number of active terms in the candidate model. The corrected Akaike Information Criterion [11, Eq. 2.2] is then defined as

$$\text{AIC}_c = \tilde{m} \ln(\overline{\text{MSE}}) + 2k + \frac{2(k+1)(k+2)}{\tilde{m} - k - 2}, \quad (2.25)$$

where the final term is a finite-sample correction that strengthens the complexity penalty when $\tilde{m}$ is small relative to k . The AIC_c balances goodness-of-fit against model complexity: increasing k typically reduces $\overline{\text{MSE}}$ but simultaneously raises the penalty term, preventing overfitting. This leads to a curve with a minimum at the Pareto frontier as illustrated in Fig. 2.1. The candidate model with the lowest AIC_c score can then be taken objectively as the identified model [11].

2.4. Evaluation Metrics

The quality of an identified model has several aspects, and no single number captures all of them. Four complementary criteria are therefore used. The first measures whether the model has the right terms, that is, the correct structure. The second measures whether those terms have the right coefficients. The third measures whether the model reproduces the right long-term behavior at the fixed points. The fourth measures whether it gives the right predictions on unseen trajectories.

2.4.1. Structural and Coefficient Accuracy

To evaluate the performance of the SINDy algorithm in recovering governing equations, it is essential to distinguish between the structural correctness of the model and the numerical precision of its estimated parameters. A comprehensive assessment is therefore conducted using the F_1 score for feature selection and the Mean Absolute Error (MAE) for coefficient accuracy.

The F_1 score measures the model's ability to correctly identify the active terms within the function library Θ . It is defined as the harmonic mean of *precision* and *recall*, based on the classification of library terms into True Positives (TP), False Positives (FP), and

False Negatives (FN) [26, Section 8.3]:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.26)$$

In the context of sparse regression, a TP occurs when a term is non-zero in both the identified matrix $\hat{\Xi}$ and the ground truth Ξ_{true} . FP represents terms incorrectly included in the model, while FN represents true physical terms that were not recognized. An F_1 score of 1.0 indicates that the identified model structure perfectly matches the underlying dynamics.

While the F_1 score captures the sparsity pattern, it does not account for the magnitude of the identified coefficients. To quantify the distance between the estimated parameters $\hat{\xi}_{i,j}$ and the true physical values $\xi_{\text{true},i,j}$, the Mean Absolute Error (MAE) of the coefficients is calculated:

$$\text{MAE}_{\Xi} = \frac{1}{l \cdot n} \sum_{i=1}^l \sum_{j=1}^n |\xi_{\text{true},i,j} - \hat{\xi}_{i,j}| \quad (2.27)$$

where l is the number of library functions and n is the number of state variables. The choice of the MAE over the RMSE is motivated by the fact that metrics based on L_1 and L_0 norms reflect the goal of sparsity more effectively than L_2 -based metrics [27, Section IV].

2.4.2. Fixed Point and Stability Analysis

A fixed point $\mathbf{x}^*$ of a dynamical system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ is a state where the system does not change over time:

$$\mathbf{f}(\mathbf{x}^*) = \mathbf{0}. \quad (2.28)$$

For the identified model $\hat{\mathbf{f}}(\mathbf{x}; \hat{\xi}) = \mathbf{\Theta}(\mathbf{x})\hat{\xi}$, the fixed points are found by solving $\hat{\mathbf{f}}(\mathbf{x}^*) = \mathbf{0}$. If the library contains polynomial terms, this reduces to finding roots of a polynomial. [28, Sec. 2.2]

The stability of a fixed point is assessed by linearising the system around it. For a one-dimensional system, the sign of $\hat{f}'$ at $\hat{x}^*$ is sufficient:

$$\hat{f}'(\hat{x}^*) = \left. \frac{d\hat{f}}{dx} \right|_{x=\hat{x}^*}. \quad (2.29)$$

If $\hat{f}'(\hat{x}^*) < 0$ the fixed point is stable, if $\hat{f}'(\hat{x}^*) > 0$ it is unstable, and if $\hat{f}'(\hat{x}^*) = 0$ higher-order terms need to be examined. [28, Sec. 2.4]

If the true equations are known, the identified fixed point $\hat{\mathbf{x}}^*$ can be compared to the true one $\mathbf{x}_{\text{true}}^*$:

$$\text{AE}_{\mathbf{x}^*} = \|\hat{\mathbf{x}}^* - \mathbf{x}_{\text{true}}^*\|_1. \quad (2.30)$$

Recovering the correct fixed points and their stability class is a strong validation of the identified model. Especially when the underlying sparsity pattern and coefficients are unknown.

2.4.3. Forward Simulation Accuracy

Beyond coefficient-based metrics and fixed point analysis, the final model's performance must be evaluated by its ability to reconstruct the system's dynamics over time. Each test trajectory is reproduced by integrating the identified model forward from its initial condition using the Forward Euler method:

$$\hat{\mathbf{x}}_{t+1} = \hat{\mathbf{x}}_t + \hat{\mathbf{f}}(\hat{\mathbf{x}}_t; \hat{\boldsymbol{\xi}}) \Delta t, \quad (2.31)$$

where $\hat{\mathbf{f}}(\hat{\mathbf{x}}_t; \hat{\boldsymbol{\xi}})$ is the identified right-hand side evaluated at the current state and Δt is the time step. The same held-out test trajectories used to compute $\overline{\text{MSE}}$ in Eq. (2.24) for model selection serve as the evaluation set here, so that the forward simulation accuracy is directly given by $\overline{\text{MSE}}$ as defined in Eqs. (2.23)–(2.24).

While $\overline{\text{MSE}}$ is used internally for the AIC_c -based model selection, it is reported as the root mean squared error

$$\overline{\text{RMSE}} = \sqrt{\overline{\text{MSE}}}, \quad (2.32)$$

which shares the units of the state variable and is therefore directly comparable to the fixed-point errors $\text{AE}_{\hat{\mathbf{x}}^*}$. The $\overline{\text{RMSE}}$ quantifies the typical deviation between the predicted and observed state per data point. A low $\overline{\text{RMSE}}$ confirms that the identified model generalises to unseen initial conditions.

3. THE HAWK-DOVE GAME

Classical game theory studies how rational decision makers behave when the outcome of each one depends on the choices of the others. In 1973, John Maynard Smith and George Price extended this idea to biology by combining it with Darwin's theory of evolution. They introduced the Hawk-Dove game, and with it gave rise to the field of evolutionary game theory [2]. Its decision makers are not rational players but a population of individuals that interact through a game, and whose reproductive success is coupled to the payoff they earn. Successful behaviours are passed on and spread, while unsuccessful ones disappear, so the composition of the population changes over time. This makes an evolutionary game a dynamical system.

A genetic algorithm can simulate the game directly, letting many individuals interact and reproduce by simple rules. A collective property of the population emerges from these microscopic interactions, and its trajectory over time is the data of interest, produced from the rules alone and not from any equation. Whether this property obeys a governing equation is unknown in advance, and this work sets out to see whether SINDy can discover one from the data.

The first part of this chapter introduces the Hawk-Dove game and the evolutionary dynamics it gives rise to. The second part then describes the genetic algorithm used to generate the data, a recent variant introduced by Laruelle et al. [3].

3.1. The Hawk-Dove Game and the Replicator Equation

The game is presented in three steps. Section 3.1.1 describes the classical two-player game and its Nash equilibrium. Section 3.1.2 follows the historical adaptation of the game to population dynamics, which leads to the replicator equation and the evolutionarily stable strategy (ESS). Section 3.1.3 then introduces a further viewpoint, a homogeneous population of mixed strategists, which is the setting needed to understand the genetic algorithm.

3.1.1. Two-Player Game and Nash Equilibrium

The Hawk-Dove game [2] models competition between two players over a shared resource of value $v > 0$. In each encounter, both players independently choose one of two actions, Hawk or Dove.

A Hawk always fights for the resource, while a Dove always yields. If a Hawk meets a Dove, the Hawk wins v and the Dove gets nothing. If both play Dove, they split the resource at random, so each gets v with probability $\frac{1}{2}$ and nothing otherwise. If both play

Hawk, they fight, and each wins v or pays the injury cost $c > 0$ with equal probability. The resulting expected payoffs are summarized in Table 3.1 [29, Tab. 1].

TABLE 3.1. EXPECTED PAYOFF MATRIX OF THE HAWK-DOVE GAME

	Hawk (H)	Dove (D)
Hawk (H)	$\frac{v-c}{2}$	v
Dove (D)	0	$\frac{v}{2}$

To determine which action is best, let $\pi(X, Y)$ denote the expected payoff of playing action X against an opponent playing action Y [30, Sec. 4.6]. The outcome depends on whether the injury cost c is smaller or larger than the resource value v .

If $c < v$, an injury costs less than the resource is worth, so fighting pays off. Even when two Hawks meet and risk injury, their expected payoff $\pi(H, H) = \frac{v-c}{2}$ is still positive, so being a Hawk beats yielding against an aggressive opponent, $\pi(H, H) > 0 = \pi(D, H)$. Against a passive opponent a Hawk takes the whole resource while a Dove only shares it, $\pi(H, D) = v > \frac{v}{2} = \pi(D, D)$. Hawk earns more whatever the opponent does, so every rational player chooses Hawk and aggression always wins.

The more interesting case is $c > v$, where an injury costs more than the resource is worth. Fighting is now risky, and the best action depends on the opponent. Against a Hawk it is better to yield, since fighting leads to an expected loss while yielding at least avoids injury, $\pi(D, H) = 0 > \frac{v-c}{2} = \pi(H, H)$. Against a Dove it is better to be aggressive, since a Hawk claims the whole resource while a Dove only shares it, $\pi(H, D) = v > \frac{v}{2} = \pi(D, D)$. No single action is a best response to itself, so neither pure action is an equilibrium and a rational player must mix the two.

Suppose a player chooses Hawk with probability p and Dove with probability $1 - p$, against an opponent using the same strategy.⁴ The expected payoffs of the two actions are

$$\pi_H = p \cdot \frac{v-c}{2} + (1-p) \cdot v \quad (3.1a)$$

$$\pi_D = p \cdot 0 + (1-p) \cdot \frac{v}{2}. \quad (3.1b)$$

Mixing is optimal only when both actions give the same expected payoff. Setting $\pi_H = \pi_D$ gives

$$p \cdot \frac{v-c}{2} + (1-p) \cdot v = (1-p) \cdot \frac{v}{2}, \quad (3.2)$$

⁴Hawk (H) and Dove (D) are *actions*, the choices made in a single encounter, whereas a *strategy* is the probability $p \in [0, 1]$ of playing Hawk.

and solving for p ,

$$p \cdot \left(\frac{v-c}{2} - v + \frac{v}{2} \right) = \frac{v}{2} - v \quad (3.3)$$

$$-\frac{cp}{2} = -\frac{v}{2}, \quad (3.4)$$

yields the Nash equilibrium [31]

$$p^* = \frac{v}{c}. \quad (3.5)$$

The equilibrium p^* is a mixed strategy, meaning the player mixes the two actions with fixed probabilities. When the opponent plays p^* , both actions give the same expected payoff, so the player is indifferent between them and cannot improve by deviating. By symmetry this holds for both players, which makes p^* a Nash equilibrium.

3.1.2. Replicator Equation and Evolutionarily Stable Strategy

The previous section analyzed the Hawk-Dove game from the viewpoint of rational players choosing a best response. Evolutionary game theory takes a different approach and instead studies how a population playing the game changes over time when reproductive success depends on the payoffs of the game in individual encounters. In the classical evolutionary interpretation, the population is heterogeneous, made up of two types of individual. Each individual is genetically programmed to play a single action, so it is either a Hawk that always plays H or a Dove that always plays D .

The total population size is held fixed, so under natural selection only the composition of the population changes. This composition is described by the frequency of Hawks p , the fraction of the population that are Hawks, while $1 - p$ is the fraction that are Doves.

Individuals meet at random in pairwise encounters and receive payoffs as described in Section 3.1.1. Over many encounters, an individual meets Hawks and Doves in proportion to their frequencies p and $1 - p$. Its average payoff is therefore π_H if it is a Hawk and π_D if it is a Dove, the same expected payoffs as in Eq. (3.1), with p now the frequency of Hawks in the population rather than a probability chosen by a player. Both depend on p , so the composition of the population decides which type earns more.

These expected payoffs determine the fitness of an individual,

$$w_H(p) = w_0 + \pi_H = w_0 + p \frac{v-c}{2} + (1-p)v, \quad (3.6)$$

$$w_D(p) = w_0 + \pi_D = w_0 + (1-p)\frac{v}{2}, \quad (3.7)$$

where $w_0 > 0$ is a baseline fitness equal for all individuals. Fitness determines reproductive success. An individual with higher fitness leaves more offspring, and each offspring is of the same type as its parent. Over time the type with the higher fitness spreads through the population, which is natural selection. The fitness of each type depends on p , so which

type is fitter depends on how common Hawks already are. This frequency-dependence is a central feature of evolutionary game theory [30, Chap. 4].

Two assumptions turn this process into a single equation. First, the population is large, so random fluctuations average out and the dynamics are deterministic. Second, time is continuous, so the composition changes smoothly rather than in discrete generations [30, Sec. 7.1]. Because fitness is a function of p , the change in p is then also a function of p . The frequency of Hawks grows when a Hawk's fitness is above the population average and falls when it is below, which is captured by the replicator equation [32],

$$\dot{p} = p(w_H - \bar{w}(p)), \quad (3.8)$$

where

$$\bar{w}(p) = p w_H(p) + (1 - p) w_D(p) \quad (3.9)$$

is the average fitness of the population. Substituting the fitness expressions and expanding gives

$$\dot{p} = \frac{1}{2}p(1-p)(v-cp) = \frac{v}{2}p - \frac{v+c}{2}p^2 + \frac{c}{2}p^3. \quad (3.10)$$

Equation (3.10) gives the rate of change of p for any composition of the population. The long-term outcome is set by its fixed points, the values of p where $\dot{p} = 0$ and the composition no longer changes. Setting $\dot{p} = 0$ gives three fixed points,

$$p = 0, \quad p = 1, \quad p^* = \frac{v}{c}. \quad (3.11)$$

From here on $c > v$ is assumed, so that $p^* = v/c \in (0, 1)$ is a biologically meaningful. The two boundary fixed points are both unstable. At $p = 0$ the population consists only of Doves, but a rare Hawk earns $\pi_H = v$ while each Dove earns $\pi_D = \frac{v}{2}$, so Hawks earn more and spread. At $p = 1$ the population consists only of Hawks, each earning $\frac{v-c}{2} < 0$, while a rare Dove earns $0 > \frac{v-c}{2}$, so Doves earn more and spread.

The only stable fixed point is the interior equilibrium $p^* = v/c$. There both types have equal fitness, $w_H(p^*) = w_D(p^*)$, so there is no selection pressure, and any small shift in either direction creates a fitness difference that pulls p back to p^* . This equilibrium is known as an evolutionarily stable strategy (ESS) [2], the strategy adopted by the population as a whole that resists invasion by any rare mutant. A rare mutant cannot spread, since whenever one type becomes more common than at p^* it earns less than the other.

Both the Nash equilibrium and the ESS give $p^* = v/c$, but for different reasons. The Nash equilibrium follows from rationality, since no player gains by deviating, whereas the ESS follows from selection, since no mutant can invade [30, Sec. 4.2–4.4]. That the two viewpoints give the same answer is a key result of evolutionary game theory.

3.1.3. Mixed Strategy Population Dynamics

Sections 3.1.1 and 3.1.2 presented two views of the Hawk-Dove game. In the game-theoretic view it is a two-player game where each player chooses a strategy $p \in [0, 1]$, the probability of playing Hawk, and the rational outcome is the Nash equilibrium. In the evolutionary view it is a large heterogeneous population made up of two types of pure strategist, Hawks and Doves, where the frequency p of Hawks changes over time under natural selection, following the replicator equation (Eq. (3.10)).

The game-theoretic view can also be taken to the population level. Instead of a single player with a mixed strategy, consider a homogeneous population of mixed strategists, with no fixed types. Every individual holds its own strategy $p_i \in [0, 1]$, playing Hawk with probability p_i and Dove with probability $1 - p_i$. The dynamics are driven by the same mechanism as in the heterogeneous case. Individuals collect payoffs in encounters, the payoffs determine fitness, and individuals with higher fitness leave more offspring, which inherit their strategy. The macroscopic state of the population is the mean Hawk strategy $\bar{p}$, and it changes over time.

Under the same idealized conditions as before, a large population and continuous time, the mean Hawk strategy $\bar{p}$ follows the same replicator equation as the frequency p . Hofbauer and Sigmund [33, Sec. 19.6] prove this equivalence, and both settings converge to the same equilibrium $p^* = v/c$.

In contrast, this setting can also be simulated directly, without these idealizations, by the genetic algorithm presented in the next section.

3.2. Simulating Evolutionary Dynamics via Genetic Algorithms

A genetic algorithm (GA) simulates the evolutionary dynamics of the Hawk-Dove game with a fixed population of N individual. In this work it provides the simulation data for the discovery, and its trajectories come only from microscopic interaction and reproduction rules and not from the replicator equation. This work follows the implementation of [3], which uses a mixed strategy in a homogeneous population as described in Section 3.1.3. At each generation t , individual i has a continuous strategy $p_i^t \in [0, 1]$ and a fitness value $w_i^t \in [0, 1]$. The strategy p_i^t is the probability that individual i plays Hawk in a given encounter, and $1 - p_i^t$ for Dove. The mean strategy of the population at generation t is

$$\bar{p}^t = \frac{1}{N} \sum_{i=1}^N p_i^t. \quad (3.12)$$

The mean Hawk strategy is the macroscopic state of interest of the system at generation t , and it evolves over time. It is stored for every generation, which gives the time trajectories that SINDy needs as input for the discovery process. The population starts from an initial state $\{(p_i^0, w_i^0)\}_{i=1}^N$, specified in Section 4.1.2, and then evolves by alternating between an interaction phase and a reproduction phase at every generation.

3.2.1. Interaction Phase

At each generation, $N_d \leq N$ individuals are chosen at random, to take part in the interaction phase to determine their fitness update. Each selected individual i plays N_e encounters against opponents chosen uniformly from the whole population, but not against themselves ($j \neq i$). The remaining $N - N_d$ individuals only participate as passive opponents, their fitness is not updated as a result of any encounter, and their fitness and strategy values carry over unchanged from the previous generation.

In each encounter, individual i plays Hawk (H) with probability p_i^t and Dove (D) with probability $1 - p_i^t$. The opponent j does the same independently with probability p_j^t and $1 - p_j^t$, respectively. Payoffs are resolved stochastically by a coin flip mechanism. The payoff received by individual i in a single encounter is summarized in Table 3.2.

TABLE 3.2. REALIZED PAYOFF $\delta w_{i,k}^t$ OF AN INDIVIDUAL IN A SINGLE ENCOUNTER.

Individual i	Opponent j	Outcome	Payoff $\delta w_{i,k}^t$
H	H	wins (prob. $\frac{1}{2}$)	$+v$
H	H	loses (prob. $\frac{1}{2}$)	$-c$
H	D	—	$+v$
D	H	—	0
D	D	takes resource (prob. $\frac{1}{2}$)	$+v$
D	D	does not take (prob. $\frac{1}{2}$)	0

Each selected individual plays N_e such encounters per generation, collecting a realized payoff $\delta w_{i,k}^t$ in each, as given by Table 3.2. After all $N_d \times N_e$ encounters, the fitness of each selected individual is updated by summing these payoffs,

$$w_i^{t+1} = w_i^t + \sum_{k=1}^{N_e} \delta w_{i,k}^t. \quad (3.13)$$

Note that the fitness update is performed only for the individual i , not for the opponents.

The accumulated payoff, and with it the fitness, can be negative. The reproduction phase, introduced in Section 3.2.2, selects individuals with probability proportional to their fitness and therefore requires non-negative values. To ensure this, the fitness of the whole population is rescaled to $[0, 1]$ by min-max normalization before reproduction,

$$w_i^{t+1} = \frac{w_i^{t+1} - \min\{w_j^{t+1}\}_{j=1}^N}{\max\{w_j^{t+1}\}_{j=1}^N - \min\{w_j^{t+1}\}_{j=1}^N} \in [0, 1], \quad (3.14)$$

where the minimum and maximum are taken over the whole population at that generation. Mapping the fitness to $[0, 1]$ keeps the birth and death probabilities well-defined and comparable across generations, regardless of the absolute payoff magnitudes.⁵ The rescaling

⁵This rescaling departs from the replicator equation, which assumes reproductive success is proportional to the absolute payoffs. Its effect on the recovered dynamics is revisited in Section 6.1.

is applied to all individuals, including those that did not take part in the encounters.

3.2.2. Reproduction Phase

In the reproduction phase, N_c individuals are replaced.⁶⁷ Each replacement event has three steps. One individual i is chosen to die, another individual $k \neq i$ is chosen to reproduce, and the offspring inherits the reproducer's strategy or is mutated.

Each of the N_c replacements pairs a death with a birth. Which individual dies is a design choice. Two versions of the genetic algorithm are run that differ only in this death rule, each producing its own dataset, as described in detail in Section 4.1.2. Which individual reproduces, by contrast, is the same in both versions. The reproducing individual k is chosen with probability proportional to its fitness,

$$P_{\text{birth}}(k) = w_k / \left(\sum_{j=1}^N w_j \right). \quad (3.15)$$

Dead individuals are not allowed to replace themselves ($i \neq k$). When an individual is chosen to reproduce, it creates an offspring (i.e. a copy of itself) that replaces the dead individual. The offspring inherits both the reproducer's strategy and fitness.

Each replacement event carries a small probability μ of mutation. When a mutation occurs, the offspring does not inherit the reproducer's strategy. Instead it receives a strategy assigned uniformly at random from $[0, 1]$, and its fitness is reset to w_0

$$p_i \sim \mathcal{U}[0, 1], \quad w_i = w_0 \quad \text{with probability } \mu. \quad (3.16)$$

Mutation allows strategies that were not present at initialization to enter over time. Once all N_c replacements, including any mutations, are complete, a new generation has been produced, differing from the previous one in N_c of its individuals. The strategies of all N individuals are then averaged using Eq. (3.12) and stored as the mean Hawk strategy $\bar{p}^t$ of this generation. Recorded over all generations, these values form the trajectory that serves as the input for the discovery.

⁶The Wright-Fisher process is the limiting case $N_c = N$. The entire population is replaced at once by multinomial sampling proportional to fitness, so generations are non-overlapping and genetic drift is high [34, Chapter IV], [35].

⁷The Moran process is the limiting case $N_c = 1$. One individual dies at random and one reproduces proportionally to fitness, with all others unchanged [36].

4. METHODOLOGY

This chapter applies SINDy to the Hawk-Dove game. The goal is to recover the governing equation of the population dynamics from trajectory data alone.

Three datasets are used, and the replicator equation plays a different role in the first dataset than in the other two. The first dataset is synthetic. Its trajectories are generated by integrating the replicator equation (Eq. (3.10)) directly, so the governing equation is known in advance. This dataset validates the pipeline by testing how much noise it can tolerate while still recovering that equation.

The second and third datasets are generated by the genetic algorithm of Section 3.2 and differ only in the microscopic death rule. Their trajectories are the mean Hawk strategy $\bar{p}'$ recorded over the generations. Here the replicator equation is not imposed, and it is not known in advance whether these dynamics admit a governing equation at all, so the replicator equation serves only as a reference for comparison. The question is if SINDy can recover a macroscopic equation from the simulation data alone, and if it detects the change in the microscopic death rule.

Section 4.1 describes the data generation and preprocessing. Section 4.2 presents the identification pipeline. Section 4.3 describes the evaluation protocol. Both the data generation and the discovery pipeline are implemented in Python⁸, using PySINDy [12], [13] for the sparse regression, SciPy [37] for numerical integration and Savitzky-Golay filtering, and NumPy [38] for array computations.

4.1. Data Generation and Preprocessing

4.1.1. Synthetic Data Generation

The synthetic dataset is generated by numerically integrating the one-dimensional replicator equation (Eq. (3.10)) with game parameters $v = 0.3$ and $c = 0.4$, which give an ESS of $p^* = v/c = 0.75$. The integration uses an adaptive Runge-Kutta method (RK45) over the time horizon $[0, 50]$, with 1000 uniformly spaced points per trajectory.

Two sets of trajectories are produced. The fitting set, used to identify the model, contains 10 trajectories with initial conditions uniformly spaced over $[0.05, 0.95]$. The held-out set, used for model selection and for evaluating the selected model, contains $\tilde{m} = 20$ trajectories with initial conditions uniformly spaced over the wider range $[0.03, 0.97]$. So, that the held-out data covers a broader region of the state space and uses initial conditions distinct from those in the fitting set.

⁸The code is available at <https://github.com/NicolasBeu/Data-Driven-Discovery-of-Dynamics-in-Evolutionary-Game-Theory.git>.

To test the robustness of the discovery process, additive zero-mean Gaussian noise is added independently to every trajectory in both sets,

$$\tilde{p}(t_i) = p(t_i) + \varepsilon(t_i), \quad \varepsilon(t_i) \sim \mathcal{N}(0, \eta^2), \quad (4.1)$$

where η is the noise standard deviation. Eleven levels are tested, $\eta \in \{0, 0.02, 0.04, \dots, 0.20\}$. Since $p \in [0, 1]$, a value of $\eta = 0.10$ corresponds to a noise level of 10%.

4.1.2. Genetic Algorithm Data Generation

The genetic algorithm produces the simulation data for the two core experiments. It follows the structure of Section 3.2 and is implemented as closely as possible to [3], with all deviations from the reference stated explicitly below. Each individual i carries a Hawk strategy $p_i^t \in [0, 1]$ and a fitness $w_i^t \in [0, 1]$. It begins from an initial state (p_i^0, w_0) , with the same initial fitness $w_0 = c = 0.4$ for all individuals and the initial strategies p_i^0 assigned as described below.

At the start of each run the population is a mixture of pure strategists. A fraction α of the N individuals are pure Hawks ($p_i^0 = 1$) and the rest pure Doves ($p_i^0 = 0$), with α set equal to the target initial mean, so that $\bar{p}^0 \approx \alpha = p_0$. Mixed strategies $p_i^t \in (0, 1)$ enter only later, through mutation.⁹

Each simulation run is initialized by such an initial mean p_0 . For the discovery process, trajectories from several initial conditions are needed, and the same p_0 values as for the synthetic data are used, 10 fitting trajectories with $p_0 \in [0.05, 0.95]$ and 20 held-out trajectories with $p_0 \in [0.03, 0.97]$, both uniformly spaced, each simulated for $G = 2,000$ generations.

The genetic algorithm is stochastic, so a single run yields only one noisy realization of the macroscopic trajectory $\bar{p}^t$. The noise is reduced in two ways. The population size is set to $N = 1,000$, larger than the $N = 100$ of the reference paper, because at the smaller size the trajectories were too noisy for reliable discovery. A larger population reduces the fluctuations in the mean strategy directly. Noise can also be reduced by averaging over repeated runs of the same initial condition. Each one is therefore simulated $n_{\text{ens}} = 40$ times independently, and these runs are averaged,

$$\langle \bar{p}^t \rangle_{n_{\text{avg}}} = \frac{1}{n_{\text{avg}}} \sum_{r=1}^{n_{\text{avg}}} \bar{p}^{(r)}(t), \quad (4.2)$$

where $\bar{p}^{(r)}(t)$ is the mean Hawk strategy at generation t in run r . It is this averaging over n_{avg} runs, rather than the population size alone, that controls how much sample-path noise remains in a trajectory. Figure 4.1 illustrates this for one initial condition and $n_{\text{avg}} = 10$.

⁹Other initializations were tested, including uniform and beta distributions over $[0, 1]$. All mixed-strategy initializations produced crossing trajectories, which violate the scalar autonomous ODE assumption of SINDy. The reference paper does not specify an initialization for a given target mean.

To quantify the effect of averaging, a sensitivity analysis over different averaging values $n_{\text{avg}} \in \{1, 5, 10, \dots, 40\}$ is performed. It also checks the reliability of the discovery, since the identified model structure should settle to a consistent result at the higher averaging levels.

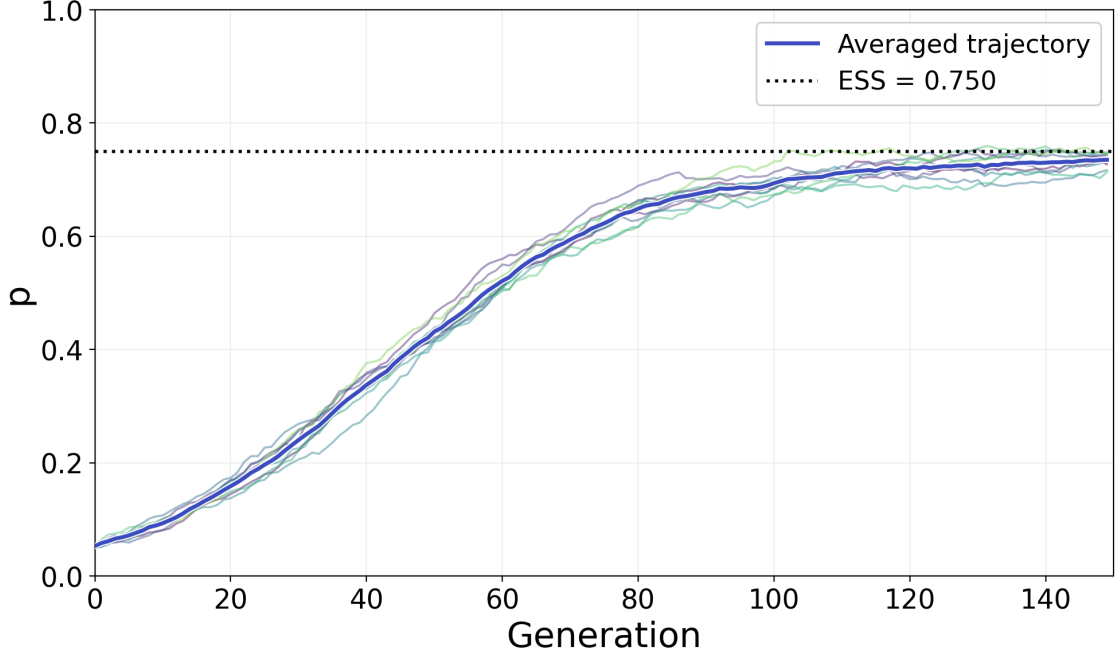


Fig. 4.1. Ensemble averaging for a single initial condition $p_0 = 0.05$ under the inverse death rule. The plot shows the 10 individual runs (faint) and their ensemble average over $n_{\text{avg}} = 10$ runs (bold blue). The dotted line marks the ESS at $p^* = 0.75$. Only the first 150 transient generations are shown.

For each run, the population evolves through an interaction phase and a reproduction phase, with the parameters of the reference paper. In the interaction phase (Section 3.2.1), $N_d = N/2 = 500$ individuals are selected at random, and each plays $N_e = N/5 = 200$ encounters against randomly chosen opponents. Payoffs are resolved as in Table 3.2, fitness accumulates across generations, and the whole population is rescaled by the min-max normalization (Eq. (3.14)) after each interaction phase.

In the reproduction phase (Section 3.2.2), N_c individuals are replaced per generation. The reference paper uses $N_c = N/5 = 200$, but here $N_c = N/20 = 50$ is used instead. Fewer replacements slow the dynamics and lengthen the transient phase where $\dot{p} \neq 0$, which is the part of the trajectory that carries the dynamical information the discovery relies on.

The two datasets that are produced by the genetic algorithm differ only in the death rule, the rule which decides which individual is going to be replaced. Keeping everything else identical isolates the effect of this single microscopic change, so any difference in the discovered macroscopic equation can be attributed to it alone.

The first dataset uses the death rule of the reference paper, under which low-fitness

individuals are more likely to die,

$$P_{\text{inverse}}(i) = \frac{1 - w_i^t}{\sum_{j=1}^N (1 - w_j^t)}, \quad (4.3)$$

where w_i^t is the rescaled fitness (Eq. (3.14)). This is the literature baseline studied in RQ2.

The second dataset replaces this with a uniform death rule, under which every individual is equally likely to die regardless of its fitness,

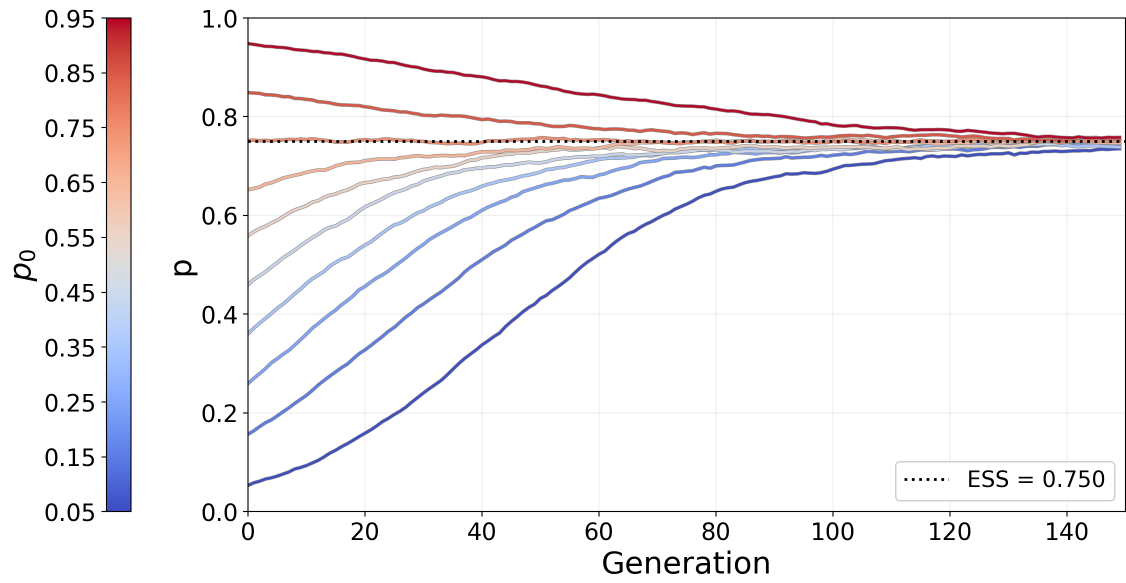
$$P_{\text{uniform}}(i) = \frac{1}{N}. \quad (4.4)$$

Comparing the two datasets tests whether the discovery process detects this microscopic change at the macroscopic level, which is the basis of RQ3.

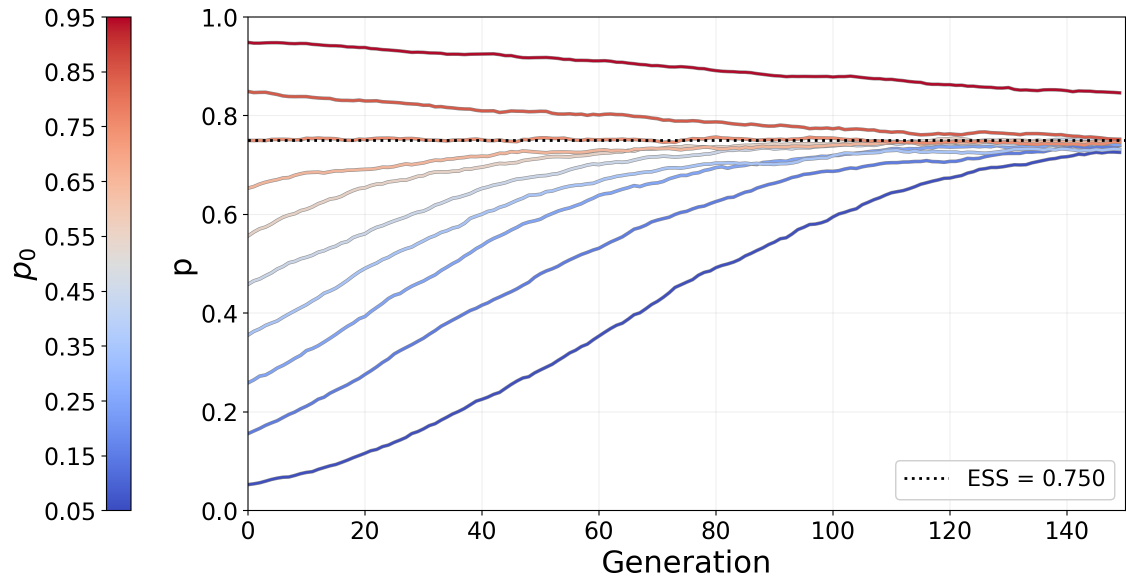
The birth and mutation rules are identical in both datasets and follow Section 3.2.2. An individual is chosen to reproduce with probability proportional to its fitness (Eq. (3.15)), and the offspring inherits the strategy and fitness of its parent. With probability $\mu = 0.005$ the offspring is instead a mutant, receiving a new strategy drawn uniformly from $[0, 1]$ and a fitness reset to w_0 . With $N_c = 50$ this gives, on average, one mutation every four generations.

In both datasets reproduction is proportional to fitness, so selection always acts through birth. Under the inverse death rule (4.3) it acts through death as well, whereas under the uniform death rule (4.4) it acts through reproduction alone.

After each generation the mean Hawk strategy $\bar{p}^t$ is recorded, giving one trajectory per run, and these are averaged as in Eq. (4.2). Figure 4.2 shows the resulting averaged trajectories for all 10 fitting initial conditions, under both death rules. All trajectories converge to the ESS at $p^* = 0.75$ and remain stable around it, consistent with evolutionary game theory and with [3]. The 20 held-out trajectories are generated in the same way. All parameter values are listed in Table 4.1, and the full procedure is given in Algorithm 4 (Appendix B).



(a) Inverse death rule



(b) Uniform death rule

Fig. 4.2. Ensemble-averaged trajectories ($n_{\text{avg}} = 10$) for all 10 fitting initial conditions, uniformly spaced in $[0.05, 0.95]$, under the inverse death rule (a) and the uniform death rule (b). Each trajectory is coloured by its initial value p_0 , from blue for low values to red for high ones, as shown by the colourbar. The trajectories approach the ESS at $p^* = 0.75$ (dotted line) from both sides, more slowly under the uniform rule, where they have not fully converged within the 150 generations shown.

TABLE 4.1. PARAMETERS FOR THE GENETIC ALGORITHM
EXPERIMENT.

Symbol	Variable / Parameter	Value
v	Benefit	0.3
c	Cost	0.4
$p^\star$	True ESS	$v/c = 0.75$
w_0	Initial fitness	$c = 0.4$
N	Population size	1,000
N_d	Active individuals per generation	$N/2 = 500$
N_e	Encounters per active individual	$N/5 = 200$
N_c	Replacements per generation	$N/20 = 50$
μ	Mutation rate	0.005
G	Generations per run	2,000
n_{ens}	Independent runs per initial condition	40
m_{fit}	Fitting trajectories	10, $p_0 \in [0.05, 0.95]$ (uniform)
$\tilde{m}$	Held-out trajectories	20, $p_0 \in [0.03, 0.97]$ (uniform)
n_{avg}	Averaged runs (sensitivity)	$\{1, 5, 10, 15, 20, 25, 30, 35, 40\}$

4.1.3. Preprocessing and Denoising

Before the discovery pipeline is applied, each trajectory is smoothed. This applies to all datasets and to both the fitting and the held-out trajectories. The fitting trajectories are smoothed for two reasons. The candidate library is more reliable when evaluated on clean data, and the time derivative $\dot{p}$ that serves as the regression target is estimated from the smoothed trajectory. The held-out trajectories are smoothed as well, since they are the target for model selection. A noisy target, compared against the model’s noise-free prediction, would make the selection less reliable. A Savitzky-Golay filter is applied to each trajectory, with the polynomial order set to the degree of the candidate library (Section 4.2.1). Its window length and the time step Δt differ between the datasets and are listed in Table 4.2.

The filter is applied only to the transient part of each trajectory. Before smoothing, each trajectory is truncated to its first T_{cut} samples. SINDy regresses $\dot{p}$ against the candidate library, but in the equilibrium region $\dot{p} \approx 0$, so the target there is essentially noise and carries no dynamical information. Keeping it would only add noise to the coefficient estimates and could promote spurious terms. For the synthetic data $T_{\text{cut}} = 1000$, the full trajectory length, since the dynamics have not fully converged by $t = 50$. For the GA data $T_{\text{cut}} = 150$ generations.

After truncation and smoothing, the fitting trajectories and their derivatives are stacked into the matrices $\mathbf{X}_{\text{fit}}$ and $\dot{\mathbf{X}}_{\text{fit}}$, as in Section 2.1. The smoothed held-out trajectories are collected as $\mathcal{P} = \{p^{(i)}\}_{i=1}^{\tilde{m}}$, where each $p^{(i)}$ is a vector of T_{cut} samples. These two collections

are the inputs to the identification pipeline (Section 4.2). All preprocessing parameters are listed together with the pipeline parameters in Table 4.2.

4.2. The Discovery Pipeline

After preprocessing, the identification pipeline is applied to the denoised data. The pipeline has three steps, used identically for the synthetic and the GA data. First, the candidate library is built. Second, a grid search over the STLSQ hyperparameters is run on the fitting data to collect candidate model structures. Third, the best model is selected from these candidates by the AIC_c criterion, evaluated on the held-out trajectories $\mathcal{P}$, which balances predictive accuracy against model complexity. The candidate collection and the model selection are each formalized as an algorithm in their subsection, and all parameters are listed in the final subsection.

Since the state variable p is one-dimensional ($n = 1$) for the homogeneous Hawk-Dove game, the SINDy coefficient matrix reduces to a vector, with $\Xi \in \mathbb{R}^{l \times n}$ becoming a coefficient vector $\xi \in \mathbb{R}^l$, and the matrices $\mathbf{X}_{\text{fit}}$ and $\dot{\mathbf{X}}_{\text{fit}}$ both are in fact column vectors.

4.2.1. Library Construction

The candidate library consists of monomials up to degree five,

$$\Theta(p) = [p, p^2, p^3, p^4, p^5]. \quad (4.5)$$

No constant term is included. The dynamics are autonomous and depend only on p , and excluding the constant ensures that $p = 0$ is a fixed point of every candidate model, consistent with an all-Dove population having no Hawks.

The library does embed one assumption, that the governing equation is a polynomial in p of degree at most five. This is a standard choice in SINDy, since a broad class of nonlinear dynamics is well approximated by low-degree polynomials [1, SI Appendix, Appendix A]. Within this polynomial family the library is not biased toward the replicator equation. The replicator terms $\{p, p^2, p^3\}$ are included, but structures that differ from it can equally be discovered.

The library is evaluated on the stacked fitting matrix $\mathbf{X}_{\text{fit}}$, giving the data library matrix $\Theta(\mathbf{X}_{\text{fit}})$ (Eq. (2.7)). Its columns are normalized to unit ℓ_2 norm before the regression, as described in Section 2.2.2. The column norms are computed once and reused across the whole grid search rather than recomputed for each fit, which speeds up the computation, and the coefficients are mapped back to the original scale afterwards.

4.2.2. Identification of Candidate Models

The candidate models are generated by running the STLSQ algorithm (Section 2.2.1) over a grid of its two hyperparameters, with a maximum of 100 iterations per run. These are the sparsity threshold $\lambda \in \Lambda$ and the ridge parameter $\alpha \in \mathcal{A}$. A larger λ removes more terms and yields a sparser model, while α adds an ℓ_2 penalty that stabilizes the regression (Eq. (2.18)). All tested values are listed in Table 4.2, and the full grid $\Lambda \times \mathcal{A}$ gives $|\Lambda| \cdot |\mathcal{A}| = 90$ fits for the synthetic data and 200 for the GA data.

Each (λ, α) combination returns a sparse coefficient vector ξ , whose support $s = \text{support}(\xi)$ is the set of active monomials. When a new support is found, it is added to the pool C , and supports already in the pool are discarded. Each newly added support is then refitted by unbiased ordinary least squares (OLS) on its active columns (Section 2.2.3), which removes the shrinkage introduced by the ridge penalty, and the coefficients are mapped back to the original scale (Eq. (2.21)). The pool C typically contains between five and ten distinct model structures. The procedure is summarized in Algorithm 2.

Algorithm 2 Find candidate models

```

1: Input:  $\mathbf{X}_{\text{fit}}, \hat{\mathbf{X}}_{\text{fit}}, \Lambda, \mathcal{A}$ 
2: Initialize: Support set  $S \leftarrow \emptyset$ , Candidate models  $C \leftarrow \emptyset$ 
3:  $\Theta \leftarrow \Theta(\mathbf{X}_{\text{fit}})$ 
4:  $\Theta_{\text{norm}} \leftarrow \Theta \mathbf{D}^{-1}$  ( $\mathbf{D} = \text{diag}(d_1, \dots, d_5)$ ,  $d_j = \|\theta_j\|_2$ )
5: for  $\lambda \in \Lambda$ ,  $\alpha \in \mathcal{A}$  do
6:    $\xi \leftarrow \text{STLSQ}(\Theta_{\text{norm}}, \hat{\mathbf{X}}_{\text{fit}}, \lambda, \alpha)$ 
7:    $s \leftarrow \text{support}(\xi)$ 
8:   if  $s \neq \emptyset$  and  $s \notin S$  then
9:      $S \leftarrow S \cup \{s\}$ 
10:     $\Theta_s \leftarrow$  columns of  $\Theta_{\text{norm}}$  indexed by  $s$ 
11:     $\tilde{\xi} \leftarrow \text{OLS}(\Theta_s, \hat{\mathbf{X}}_{\text{fit}})$ 
12:     $\hat{\xi} \leftarrow \mathbf{D}_s^{-1} \tilde{\xi}$ 
13:     $C \leftarrow C \cup \{(s, \hat{\xi})\}$ 
14:   end if
15: end for
16: Return:  $C$ , where each  $(s, \hat{\xi}) \in C$  defines a model  $\hat{f}(p; \hat{\xi}) = \sum_{j \in s} \hat{\xi}_j \theta_j(p)$ 

```

4.2.3. Candidate Ranking with AIC_c

Each candidate $(s, \hat{\xi}) \in C$ is evaluated on the $\tilde{m} = 20$ held-out trajectories $\mathcal{P}$. The model is simulated forward in time using the Forward Euler method (Eq. (2.31)), with $\Delta t \approx 0.050$ for the synthetic data and $\Delta t = 1$ generation for the GA data. Each simulation starts from the initial condition of a held-out trajectory $p^{(i)} \in \mathcal{P}$ and runs for T_{cut} steps, matching the length of the truncated trajectories.

MSE_i is the mean squared error between the simulated and the smoothed held-out trajectory (Eq. (2.23)). If a simulated trajectory leaves the physically valid range ($\hat{p} < 0$ or

$\hat{p} > 1$), it is instead assigned a penalty $\text{MSE}_i = 1$. The mean $\overline{\text{MSE}}$ over the $\tilde{m}$ trajectories (Eq. (2.24)) then gives the AIC_c score of each candidate through Eq. (2.25), where $k = |s|$ is the number of active terms. The candidate with the lowest AIC_c is returned as the final model $(s_{\text{best}}, \hat{\xi}_{\text{best}})$. The procedure is summarized in Algorithm 3.

Algorithm 3 Model selection

```

1: Input:  $C, \mathcal{P}, T_{\text{cut}}, \Delta t$ 
2: for each  $(s, \hat{\xi}) \in C$  do
3:   for  $i = 1$  to  $\tilde{m}$  do
4:      $\hat{p}_0 \leftarrow p^{(i)}[0]$ 
5:     Simulate  $\hat{p}_{t+1} \leftarrow \hat{p}_t + \hat{f}(\hat{p}_t; \hat{\xi}) \Delta t$  for  $T_{\text{cut}}$  steps from  $\hat{p}_0$ 
6:     if  $\hat{p}_t \in [0, 1]$  for all  $t$  then
7:       Compute  $\text{MSE}_i$  against  $p^{(i)}$ 
8:     else
9:        $\text{MSE}_i \leftarrow 1$ 
10:    end if
11:  end for
12:   $\overline{\text{MSE}} \leftarrow \frac{1}{\tilde{m}} \sum_{i=1}^{\tilde{m}} \text{MSE}_i$ 
13:   $k \leftarrow |s|$ 
14:  Compute  $\text{AIC}_c(k, \overline{\text{MSE}})$  for candidate  $(s, \hat{\xi})$ 
15: end for
16: Return:  $(s_{\text{best}}, \hat{\xi}_{\text{best}}) \leftarrow \arg \min_{(s, \hat{\xi}) \in C} \text{AIC}_c(s, \hat{\xi})$ 

```

4.2.4. Pipeline Parameters

All parameters of the data preprocessing and the discovery pipeline are listed in Table 4.2.

TABLE 4.2. PARAMETERS OF THE SINDY DISCOVERY PIPELINE.

Parameter	Description	Synthetic	GA Data
<i>Preprocessing</i>			
W	Savitzky-Golay window (points)	51	21
d	Savitzky-Golay polynomial order	5	5
Δt	Time step size	0.050	1
T_{cut}	Trajectory cutoff (steps)	1000	150
<i>STLSQ Optimizer</i>			
Library	Candidate functions	$\{p, p^2, p^3, p^4, p^5\}$	$\{p, p^2, p^3, p^4, p^5\}$
$\lambda \in \Lambda$	Sparsity threshold (log-spaced)	$[10^{-2}, 10^0]$ (30)	$[10^{-3}, 10^{-0.3}]$ (50)
$\alpha \in \mathcal{A}$	ℓ_2 regularization	$\{0, 10^{-2}, 10^{-1}\}$	$\{0, 10^{-3}, 10^{-2}, 10^{-1}\}$
R	Maximum STLSQ iterations	100	100
Total fits	$ \Lambda \cdot \mathcal{A} $	90	200

4.3. Evaluation Protocol

In all cases the replicator equation serves as the benchmark for the structure-, coefficient-, and fixed-point-based metrics. For the synthetic data this is immediate, since the trajectories are generated from it directly. For the GA data the justification comes from Section 3.1.3. The predictive error is the only metric that does not use this benchmark, since it measures the fit of the model to the held-out trajectories directly.

The evaluation has two parts. First, the model-selection process is examined by comparing the candidate models against each other with AIC_c . Second, the selected model is evaluated in detail across four criteria. Both parts are carried out at every noise level for the synthetic data and at every averaging level for the GA data, so the same procedure also shows how robust the result is to noise and whether it converges to a consistent model as more runs are averaged. The latter matters for the GA data, where the true governing equation is not directly known.

The held-out trajectories serve a dual role throughout. They are used by the AIC_c criterion to select the model, and the predictive error of the selected model is then reported on the same trajectories. The reported error is therefore not measured on fully independent data.

4.3.1. Evaluation of the Selection Process

The full discovery pipeline, which consists of preprocessing, candidate model generation, and AIC_c -based selection, is run independently for each noise level (synthetic data) and each averaging level n_{avg} (GA data), as listed in Section 4.1.1 and Table 4.1.

For each run, every model in the candidate pool C is assigned an AIC_c score based on its number of active terms k and its predictive accuracy $\overline{MSE}$ on the 20 held-out trajectories (Eq. (2.25)). The model with the lowest score is selected as the best model.

To examine how decisive the selection was, the AIC_c difference between the best and the second-best model is reported,

$$\Delta AIC_{c,2} = AIC_{c,2} - AIC_{c,\text{best}}. \quad (4.6)$$

A large value indicates that the best model is clearly preferred over all alternatives. The structure of the second-best model is also recorded.

In addition, it is checked whether the true Replicator structure $\{p, p^2, p^3\}$ was found by the STLSQ optimizer. If it was not found, it is labelled as not found and added manually to the candidate pool so that its AIC_c can still be evaluated. The quantity

$$\Delta AIC_{c,\text{Rep}} = AIC_{c,\text{Rep}} - AIC_{c,\text{best}} \quad (4.7)$$

measures how much worse the Replicator structure describes the data compared to the best model. If the best model is the Replicator, this value is zero by definition.

4.3.2. Evaluation of the Best Model

For each noise or averaging level, the full identified equation of the best model is reported. The model is then evaluated across four criteria.

The first criterion is the sparsity pattern. The identified support is compared to the true Replicator support $\{p, p^2, p^3\}$ using the F_1 score (Eq. (2.26)). A value of $F_1 = 1.0$ means the identified sparsity pattern matches the Replicator equation exactly.

The second criterion is the coefficient accuracy. For the synthetic data, the identified coefficients are compared to the true values via MAE_{Ξ} (Eq. (2.27)). This metric is computed over the full library, so it is non-zero whenever the identified coefficients differ from the theoretical replicator values, in particular when terms outside the replicator structure are present. For the GA data this evaluation is not possible, since the GA trajectories are scaled by an effective timescale that depends on N_c and other algorithm parameters, so the true coefficient values are unknown.

The third criterion is the fixed point analysis. By construction, $p = 0$ is always a fixed point of any model in the library, since no constant term is included. The remaining fixed points are computed as the real roots of $\hat{f}(p; \hat{\xi}) = 0$. The interior fixed point closest to the true ESS p^* is taken as the model's ESS estimate and denoted $\hat{p}^*$. The fixed point closest to $p = 1$ is labelled $\hat{r}_1$, and any additional fixed points are reported as well.

The ESS estimate $\hat{p}^*$ is one of the most important outcomes of this analysis, as it directly quantifies how well the identified model captures the equilibrium of the system. For the GA data the true ESS is known ($p^* = v/c$), so $\hat{p}^*$ can still be compared even though the individual coefficients cannot. The absolute errors are

$$\text{AE}_{\hat{p}^*} = |\hat{p}^* - p^*|, \quad \text{AE}_{\hat{r}_1} = |\hat{r}_1 - 1|. \quad (4.8)$$

Both are absolute differences in p . The stability of each fixed point is classified following Eq. (2.29). For the true Replicator equation, the ESS p^* is stable and $p = 1$ is unstable.

The fourth criterion is the predictive accuracy. The $\overline{\text{RMSE}}$ (Eq. (2.32)) on the held-out trajectories is reported as the final predictive error.

5. RESULTS

This chapter presents the results for the three datasets, the synthetic data (Section 5.1), the genetic-algorithm data with the inverse-proportional death rule (Section 5.2), and the genetic-algorithm data with the uniform death rule (Section 5.3). Each section answers one of the research questions stated in the Introduction.

The synthetic section has a different purpose from the two genetic-algorithm sections. There the trajectories are generated from the replicator equation, so it is the known ground truth and the section asks how much noise the pipeline tolerates while still recovering it. For the genetic-algorithm data no governing equation is known beforehand, so each section opens with the equation the pipeline identifies at the highest average level and uses the replicator only as a reference.

Despite this difference, the three sections are organized in the same way. At each noise or averaging level, each reports the identified equation and the model-selection statistics behind it, then evaluates that equation through its structure, the dynamics set by its fixed points, and its prediction on the held-out trajectories. The sensitivity analysis then tracks these results across all levels, and the full evaluation tables are given in Appendix C.

5.1. RQ1: Pipeline Design and Robustness to Noise (Synthetic Data)

The synthetic data were generated from the replicator equation (Eq. (3.10)) with game parameters $v = 0.3$ and $c = 0.4$, which in the monomial basis reads

$$\dot{p} = 0.15 p - 0.35 p^2 + 0.20 p^3. \quad (5.1)$$

This is the equation the pipeline should recover, so it serves as the reference for the identified models in Table 5.1, whose coefficients can be compared to it term by term. The pipeline recovers it almost exactly up to a noise level of $\eta = 14\%$. Below this threshold the identified structure matches the replicator at every noise level, with a perfect structure score $F_1 = 1.000$ and a coefficient error MAE_{Ξ} below 7×10^{-3} .

The model-selection results for all noise levels are collected in Table 5.1. For each noise level with $\eta \leq 14\%$ the replicator equation is the selected model and is always found directly by the optimizer. The replicator is preferred over the next-best candidate at every noise level, by a margin of 4.5 to 38.4 AIC_c units. The selection is therefore clear. At $\eta \geq 16\%$ the selection changes. The preferred structure becomes $\{p, p^2, p^4, p^5\}$, and the replicator equation, although still found, now trails it by more than 40 AIC_c units. This means that beyond this point the noise changes the structure of the recovered equation, not just its coefficients.

TABLE 5.1. SYNTHETIC DATA - MODEL SELECTION

η (%)	Identified equation ($\dot{p} =$)	AIC _c	2nd-best structure	$\Delta\text{AIC}_{c,2}$	Rep. found	$\Delta\text{AIC}_{c,\text{Rep}}$
0	$0.150p - 0.351p^2 + 0.201p^3$	-303.36	$\{p, p^2, p^3, p^4, p^5\}$	4.53	✓	0
2	$0.149p - 0.345p^2 + 0.195p^3$	-191.49	$\{p, p^3, p^4, p^5\}$	26.75	✓	0
4	$0.149p - 0.340p^2 + 0.191p^3$	-164.63	$\{p, p^2, p^3, p^4, p^5\}$	33.15	✓	0
6	$0.148p - 0.336p^2 + 0.187p^3$	-149.16	$\{p, p^2, p^3, p^4, p^5\}$	38.44	✓	0
8	$0.148p - 0.334p^2 + 0.186p^3$	-138.12	$\{p, p^2, p^4, p^5\}$	12.44	✓	0
10	$0.148p - 0.334p^2 + 0.186p^3$	-129.31	$\{p, p^2, p^4, p^5\}$	13.33	✓	0
12	$0.149p - 0.336p^2 + 0.188p^3$	-121.56	$\{p, p^2, p^4, p^5\}$	13.49	✓	0
14	$0.150p - 0.341p^2 + 0.192p^3$	-113.71	$\{p, p^2, p^4, p^5\}$	12.16	✓	0
	$0.242p - 0.616p^2$					
16	$+ 1.066p^4 - 0.709p^5$	-96.09	$\{p, p^2, p^3, p^4, p^5\}$	7.01	✓	45.84
	$0.252p - 0.648p^2$					
18	$+ 1.130p^4 - 0.749p^5$	-91.41	$\{p, p^2, p^3, p^4, p^5\}$	4.45	✓	41.53
	$0.258p - 0.665p^2$					
20	$+ 1.155p^4 - 0.760p^5$	-87.28	$\{p, p^2, p^3, p^4, p^5\}$	4.15	✓	50.63

To show how the identified model and the data correspond, Figure 5.1 plots the predictions at $\eta = 8\%$. The model is integrated forward from four initial conditions and it can be seen that it tracks the held-out trajectories closely, approaching the ESS from both below and above.

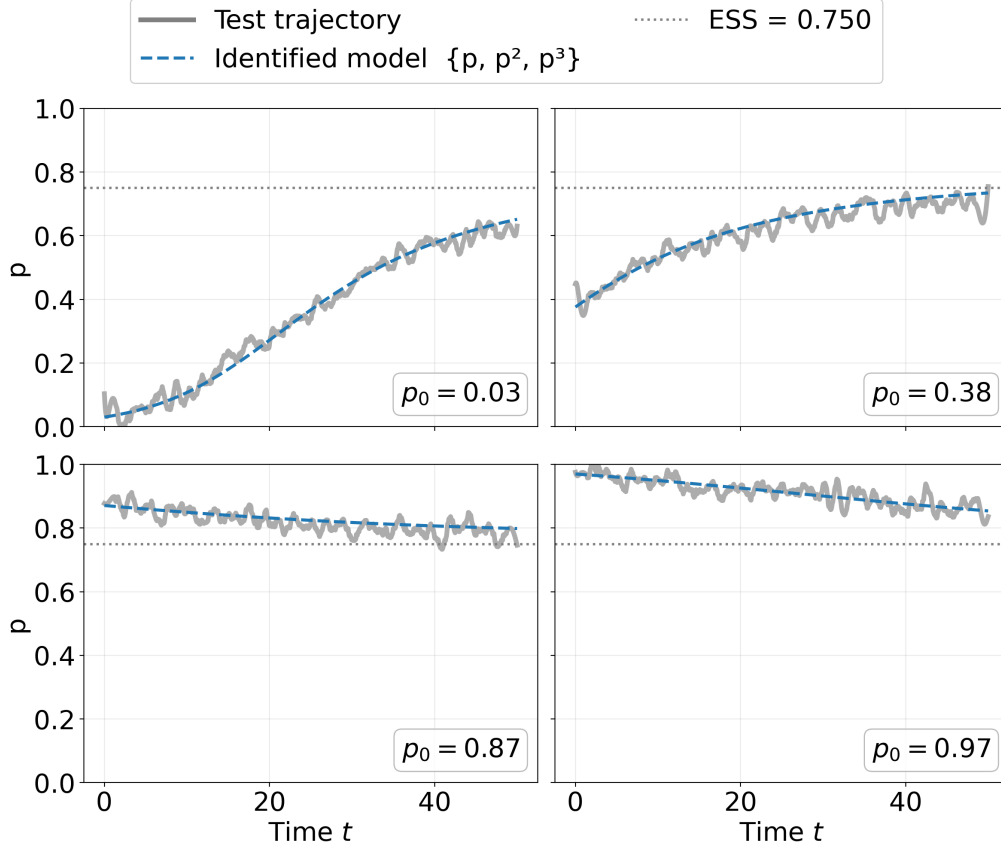


Fig. 5.1. Trajectory predictions for the synthetic dataset at noise level $\eta = 8\%$. Each panel corresponds to one held-out trajectory and shows the smoothed noisy data (grey) together with the forward prediction of the identified model (blue dashed), for four representative initial conditions p_0 . Two starting below the ESS and two above it. At this noise level the identified governing equation is $\dot{p} = 0.148 p - 0.334 p^2 + 0.186 p^3$. The dotted line marks the ESS at $p^* = 0.75$. The predicted trajectories follow the data closely and approach the ESS from both directions.

Beyond the recovery threshold the pipeline no longer selects the replicator equation. At $\eta \geq 16\%$ it switches to the structure $\{p, p^2, p^4, p^5\}$, which still fits the held-out trajectories almost as well. The trajectory error $\overline{\text{RMSE}}$, which measures the fit to the data and not agreement with the replicator, keeps rising only gradually and shows no clear break at the threshold, so on the data alone the new equation is not clearly worse. Its dynamics, however, are no longer those of the game. The structure score drops to 0.571, the interior fixed point drifts away from the ESS to between 0.895 and 0.927 with an error above 0.14, and the boundary point moves outside $[0, 1]$ to about -0.76 , where it turns stable, while two complex fixed points appear. The equation still tracks the data, but its equilibria are wrong, so it describes a different system. Figure 5.2 shows this across all noise levels. Up to $\eta = 14\%$ the structure score stays at one and every error grows only gradually. At $\eta = 16\%$ the structure changes and every metric measured against the replicator follows, with the coefficient error jumping by more than two orders of magnitude and the fixed-point errors rising sharply, while $\overline{\text{RMSE}}$ alone shows no such break. The threshold at

$\eta = 14\%$ is therefore a limit on recovering the correct dynamics, not on fitting the data.

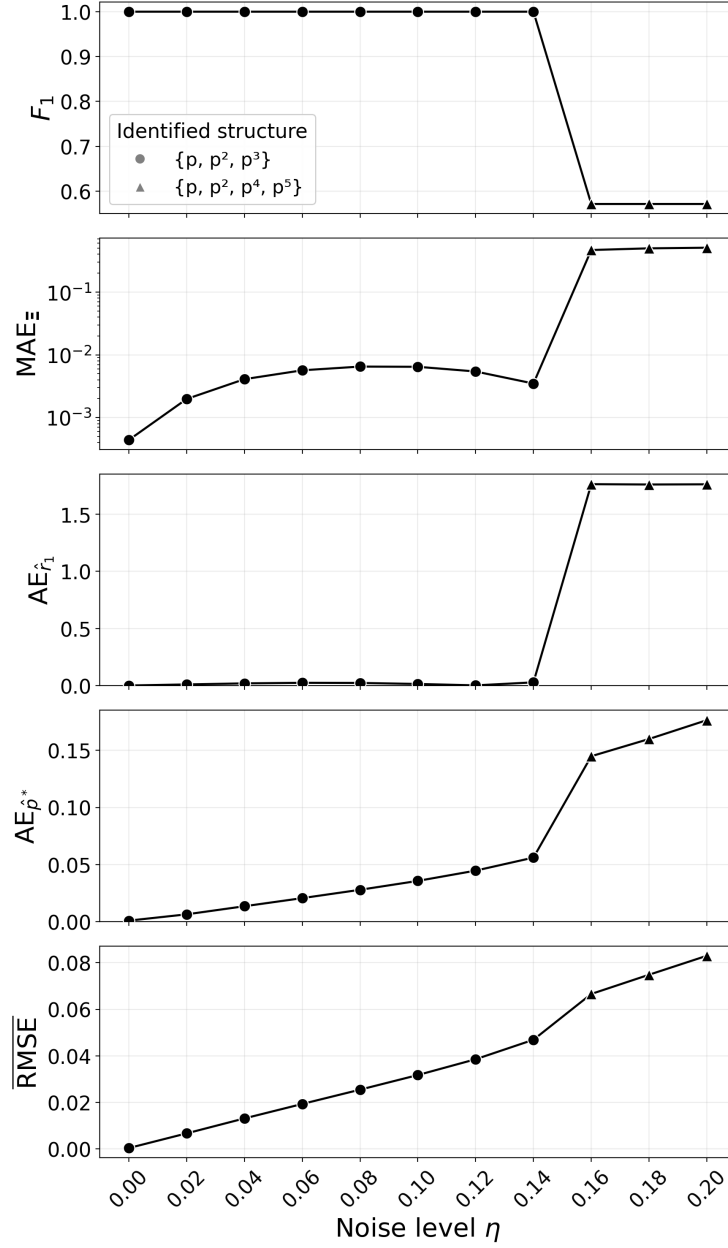


Fig. 5.2. Recovery quality for the synthetic dataset as a function of the noise level η . From top to bottom the panels show the F_1 score, the coefficient error MAE_{Ξ} (log scale), the boundary fixed-point error AE_{r_1} , the ESS error AE_{p^*} , and the trajectory error $\overline{RMSE}$. Circles denote the identified structure $\{p, p^2, p^3\}$, which is the replicator, and triangles denote $\{p, p^2, p^4, p^5\}$. The replicator is recovered up to $\eta = 14\%$, where the F_1 score stays at one and the errors grow only slowly. At $\eta \geq 16\%$ the identified structure changes and every metric degrades, marking the breakdown of the identification.

Overall, the pipeline reliably recovers the replicator equation from noisy synthetic data. Up to $\eta = 14\%$ it recovers the correct structure, coefficients, and fixed-point picture, and only beyond this level does the identification break down. This establishes the noise range over which the pipeline recovers a known equation, before it is applied to the

genetic-algorithm data.

5.2. RQ2: The Literature Baseline (Inverse Death Rule)

The genetic algorithm with the inverse-proportional death rule produces dynamics that are best described not by the replicator equation but by the structure $\{p, p^3, p^4\}$. At $n_{\text{avg}} = 40$ the identified model is

$$\dot{p} = 0.047 p - 0.196 p^3 + 0.149 p^4. \quad (5.2)$$

Although this structure differs from the replicator, it reproduces the same qualitative equilibrium picture. It has a stable interior fixed point at $\hat{p}^* = 0.754$ that recovers the ESS with high accuracy ($\text{AE}_{\hat{p}^*} = 0.004$) and an unstable boundary fixed point at $\hat{r}_1 = 0.986$ close to 1 ($\text{AE}_{\hat{r}_1} = 0.014$).

The model-selection results across the averaging sweep are collected in Table 5.2. For this dataset the replicator structure $\{p, p^2, p^3\}$ is never produced by the optimizer, so it is added to the candidate pool by hand and scored alongside the discovered structures. At $n_{\text{avg}} = 1$ the single unaveraged run is noisy enough that the replicator is selected, ahead of the second-best candidate by a margin of 19.21. From $n_{\text{avg}} = 5$ onward the structure $\{p, p^3, p^4\}$ is consistently preferred, with margins over the second-best between 13.08 and 22.89 and margins over the replicator between 13.08 and 24.87. The selection for $n_{\text{avg}} \geq 5$ is very clear. The replicator selection at $n_{\text{avg}} = 1$ is therefore an artifact of the single-run noise, and the averaged data is better described by the other structure $\{p, p^3, p^4\}$.

TABLE 5.2. GA DATA (INVERSE DEATH) - MODEL SELECTION

n_{avg}	Identified equation ($\dot{p} =$)	AIC _c	2nd-best structure	$\Delta\text{AIC}_{c,2}$	Rep. found	$\Delta\text{AIC}_{c,\text{Rep}}$
1	$0.081p - 0.178p^2 + 0.094p^3$	-139.49	$\{p, p^2\}$	19.21	×	0
5	$0.048p - 0.202p^3 + 0.155p^4$	-167.67	$\{p, p^2, p^3\}$	13.08	×	13.08
10	$0.048p - 0.199p^3 + 0.152p^4$	-177.54	$\{p, p^2, p^3, p^4\}$	14.36	×	19.38
15	$0.047p - 0.196p^3 + 0.150p^4$	-179.15	$\{p, p^2, p^3\}$	20.08	×	20.08
20	$0.048p - 0.197p^3 + 0.150p^4$	-182.03	$\{p, p^2, p^3\}$	22.89	×	22.89
25	$0.047p - 0.196p^3 + 0.149p^4$	-180.44	$\{p, p^2, p^3, p^4\}$	20.83	×	21.42
30	$0.048p - 0.196p^3 + 0.149p^4$	-183.36	$\{p, p^2, p^3, p^4\}$	22.34	×	24.87
35	$0.047p - 0.196p^3 + 0.149p^4$	-182.64	$\{p, p^2, p^3, p^4\}$	22.05	×	24.25
40	$0.047p - 0.196p^3 + 0.149p^4$	-182.04	$\{p, p^2, p^3, p^4\}$	20.85	×	22.99

The fixed points of the selected model, reported in full in the appendix (Table C.2), confirm that this non-replicator structure reproduces the same equilibrium as the replicator. For $n_{\text{avg}} \geq 5$ the ESS estimate $\hat{p}^*$ stays between 0.753 and 0.755 with an absolute error of at most 0.005, and the boundary point $\hat{r}_1$ remains close to 1 and unstable. The structure $\{p, p^3, p^4\}$ does carry a third fixed point $\hat{r}_2$, but it is negative and lies outside $[0, 1]$, so it does not affect the dynamics. The discovered equation therefore shares the

equilibrium picture of the game, a stable ESS and an unstable boundary, even though its structure differs from the replicator.

Figure 5.3 compares the predictive capacity of the identified model $\{p, p^3, p^4\}$ and the replicator equation against the held-out trajectories at $n_{\text{avg}} = 40$. The replicator equation is not found by the optimizer on this data, so it is fitted to the data by ordinary least squares on its own structure and added for comparison. The selected model and the replicator are integrated forward from four initial conditions and shown together with the data. The identified model tracks the trajectories more closely, while the replicator deviates most visibly for the lowest and highest starting points.

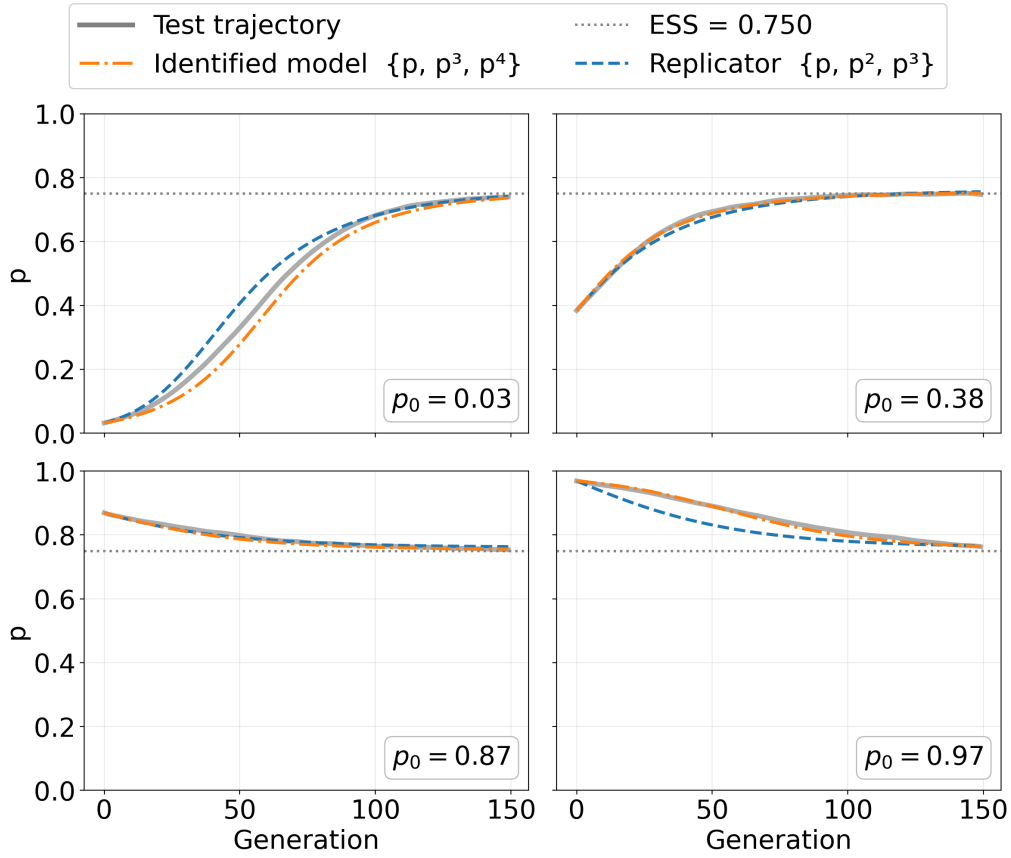


Fig. 5.3. Trajectory predictions for the genetic-algorithm data with the inverse-proportional death rule, at $n_{\text{avg}} = 40$. Each panel shows one held-out trajectory (gray) together with the forward prediction of the identified model (orange dash-dot) and of the replicator model (blue dashed), for four representative initial conditions p_0 . Two below the ESS and two above it. The identified governing equation is $\dot{p} = 0.047 p - 0.196 p^3 + 0.149 p^4$, whose structure $\{p, p^3, p^4\}$ differs from the replicator form $\{p, p^2, p^3\}$. The dotted line marks the ESS at $p^* = 0.75$. The identified model follows the data more closely than the replicator, most visibly for the extreme starting points, while both share the same equilibrium.

Figure 5.4 follows the recovery metrics across the whole averaging sweep, again with the replicator equation in blue included for comparison.

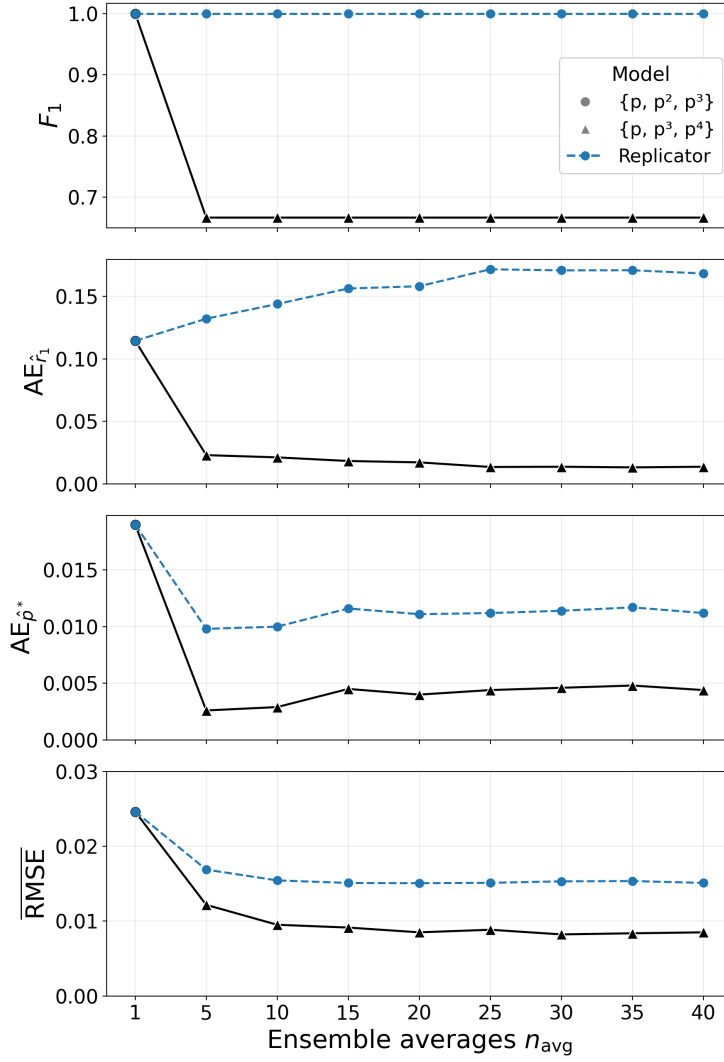


Fig. 5.4. Recovery quality for the genetic-algorithm data under the inverse-proportional death rule, as a function of the number of ensemble averages n_{avg} . From top to bottom the panels show the F_1 score, the boundary fixed-point error $AE_{\hat{r}_1}$, the ESS error $AE_{\hat{p}^*}$, and the trajectory error $RMSE$. The identified model is shown in black, with markers encoding its structure (circles for $\{p, p^2, p^3\}$, triangles for $\{p, p^3, p^4\}$), and the replicator model in blue for comparison. At $n_{\text{avg}} = 1$ the single noisy run selects the replicator structure, so the identified and replicator models coincide. From $n_{\text{avg}} \geq 5$ the identification settles on $\{p, p^3, p^4\}$, which lowers all three error measures below those of the replicator despite its lower F_1 score, and the metrics stabilize.

The change of structure between $n_{\text{avg}} = 1$ and $n_{\text{avg}} = 5$ is accompanied by a sharp drop in the ESS error and the trajectory error, after which both stabilise and improve only marginally up to $n_{\text{avg}} = 40$. The replicator equation has $F_1 = 1$ by construction, since the score is measured against its own structure. But the selected model $\{p, p^3, p^4\}$ reaches a lower boundary error, ESS error, and trajectory error at every averaging level. This confirms that the structure $\{p, p^3, p^4\}$ describes the data generated by the inverse-proportional death rule of [3] more accurately than the replicator equation.

5.3. RQ3: Microscopic Sensitivity (Uniform Death Rule)

The discovery process detects the change in the microscopic death rule at the macroscopic level. While the inverse death rule was described by the non-replicator structure $\{p, p^3, p^4\}$ (Section 5.2), the uniform death rule recovers the replicator structure $\{p, p^2, p^3\}$ with $F_1 = 1.000$ at every averaging level and converges to a stable result as n_{avg} increases. At $n_{\text{avg}} = 40$ the identified model is

$$\dot{p} = 0.059p - 0.130p^2 + 0.070p^3, \quad (5.3)$$

which has the same structure as the replicator equation and reproduces its fixed points with small errors.¹⁰

The model-selection results are collected in Table 5.3. The replicator equation is the selected structure at every averaging level, with $\Delta\text{AIC}_{c,\text{Rep}} = 0$ throughout, and it is preferred over the second-best candidate by a wide margin, between 28.10 and 50.20. At $n_{\text{avg}} = 1$ the replicator is not produced by the optimizer and is added to the candidate pool by hand, meaning that without averaging, noise blocks the optimizer from finding the structure on its own. But from $n_{\text{avg}} = 5$ onward it is found directly. The selection is therefore unambiguous across all $n_{\text{avg}} \geq 5$.

TABLE 5.3. GA DATA (UNIFORM DEATH) - MODEL SELECTION

n_{avg}	Identified equation ($\dot{p} =$)	AIC_c	2nd-best structure	$\Delta\text{AIC}_{c,2}$	Rep. found	$\Delta\text{AIC}_{c,\text{Rep}}$
1	$0.062p - 0.139p^2 + 0.077p^3$	-124.50	$\{p, p^2\}$	28.10	×	0
5	$0.059p - 0.132p^2 + 0.073p^3$	-148.31	$\{p, p^2\}$	50.20	✓	0
10	$0.057p - 0.127p^2 + 0.068p^3$	-150.81	$\{p, p^2\}$	48.07	✓	0
15	$0.057p - 0.127p^2 + 0.068p^3$	-150.00	$\{p, p^2\}$	46.07	✓	0
20	$0.058p - 0.128p^2 + 0.069p^3$	-147.45	$\{p, p^2\}$	42.20	✓	0
25	$0.058p - 0.129p^2 + 0.070p^3$	-147.59	$\{p, p^2\}$	42.89	✓	0
30	$0.058p - 0.129p^2 + 0.070p^3$	-147.88	$\{p, p^2\}$	42.99	✓	0
35	$0.059p - 0.131p^2 + 0.071p^3$	-148.64	$\{p, p^2\}$	43.68	✓	0
40	$0.059p - 0.130p^2 + 0.070p^3$	-148.74	$\{p, p^2\}$	43.16	✓	0

The fixed points of the identified model, reported in full in the appendix (Table C.3), match those of the replicator equation up to an error. The interior fixed point $\hat{p}^*$ is stable and the boundary point $\hat{r}_1$ is unstable at every averaging level, and the trajectory error $\overline{\text{RMSE}}$ stays below 3.6×10^{-2} . The ESS estimate is biased upward. It lies above the theoretical value $p^* = 0.75$ at every level, between 0.775 and 0.786, so its absolute error $\text{AE}_{\hat{p}^*}$ sits between 0.025 and 0.036. The boundary point $\hat{r}_1$ likewise lies just above 1, between 1.029 and 1.080, with an error $\text{AE}_{\hat{r}_1}$ between 0.029 and 0.080. Apart from this bias, the identified model follows the same dynamics as the replicator.

¹⁰Only the structure and the fixed points are compared here, not the coefficient magnitudes. This is discussed in Section 6.2.

Figure 5.5 shows the model predictions at $n_{\text{avg}} = 40$. The identified model is integrated forward from four initial conditions and tracks the held-out trajectories closely, approaching the ESS from both directions. Note that there is a visible departure at the highest starting point, where the model converges a little bit faster than the data.

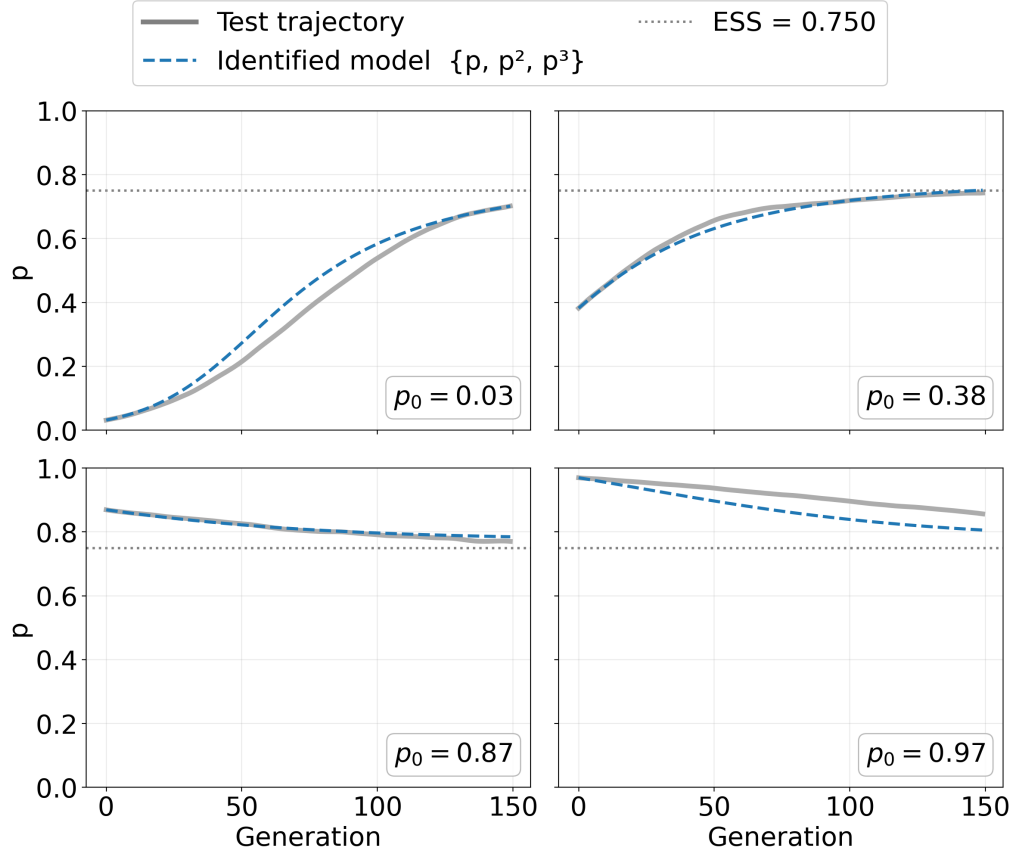


Fig. 5.5. Trajectory predictions for the genetic-algorithm data under the uniform death rule, at $n_{\text{avg}} = 40$. Each panel shows one held-out trajectory (grey) together with the forward prediction of the identified model (blue dashed), for four representative initial conditions p_0 . Two below the ESS and two above it. The identified model recovers the replicator structure $\{p, p^2, p^3\}$. The dotted line marks the ESS at $p^* = 0.75$. The predicted trajectories follow the data closely and approach the ESS from both directions, with a small deviation visible for the highest starting point, where the model converges a little bit faster than the data.

Figure 5.6 follows the recovery metrics across the averaging sweep.

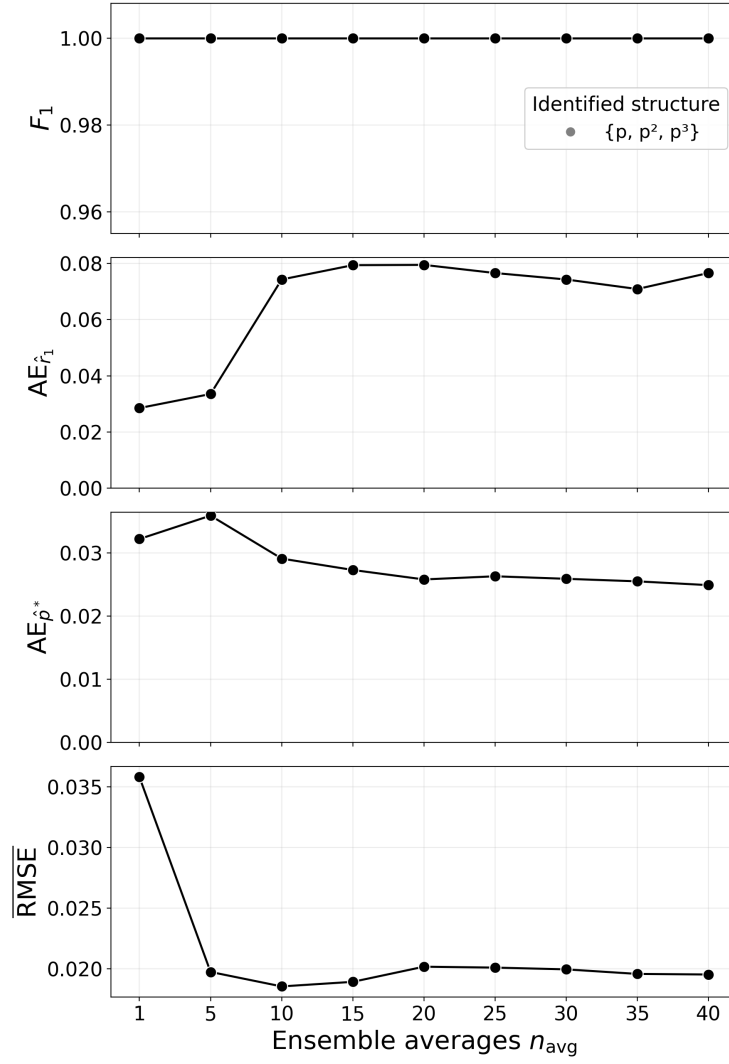


Fig. 5.6. Recovery quality for the genetic-algorithm data under the uniform death rule, as a function of the number of ensemble averages n_{avg} . From top to bottom the panels show the F_1 score, the boundary fixed-point error $AE_{\hat{\rho}_1}$, the ESS error $AE_{\hat{\rho}^*}$, and the trajectory error $\overline{\text{RMSE}}$. The identified structure is the replicator form $\{p, p^2, p^3\}$ at every averaging level (circles), so the F_1 score stays at one throughout. The trajectory error is largest for the single run and drops sharply once a few runs are averaged, after which all measures stabilise. The ESS error settles around 0.025, reflecting a small systematic overshoot of the estimated ESS above 0.75.

The F_1 score stays at one throughout, confirming that the replicator structure is identified at every level. The trajectory error is largest for the single unaveraged run and drops sharply once a few runs are combined, after which all measures stabilise. As noted above, the ESS error and the boundary error reflect the upward bias of both fixed points. Taken together, the data generated by the genetic algorithm with the uniform death rule is best described by the replicator equation, in contrast to the non-replicator structure recovered under the inverse death rule. The discovery therefore distinguishes the two microscopic rules through the macroscopic equation it finds.

6. DISCUSSION

6.1. Interpretation of the Results

The discovery pipeline, which combines Savitzky-Golay denoising, sparse regression by sequentially thresholded least squares, and model selection by the corrected Akaike information criterion, performed well across all three datasets. Validating it first on synthetic data, where the equation is known, fixed the noise level up to which a known equation is recovered before any unknown data was touched. On the genetic-algorithm data the discovered equation was stable for most $n_{\text{avg}} \geq 5$, so within the chosen library the findings rest on a solid basis. The main result is that changing the microscopic death rule changes the macroscopic equation that the pipeline recovers. The uniform rule yields the replicator structure $\{p, p^2, p^3\}$ and the inverse rule the structure $\{p, p^3, p^4\}$, and both share the same equilibrium, so the death rule changes the form of the equation, not where the population settles.

The structural difference can be motivated by the analysis of Traulsen and Hauert [39], who study a stochastic birth-death process known as the Moran process. In this process one individual dies at random and is replaced by the offspring of a parent that is selected with a probability proportional to its fitness. The genetic algorithm used here differs from this process in several ways, since it replaces $N_c = 50$ individuals per generation rather than one and rescales the fitness. Therefore the following gives only a reason for the result rather than a rigorous argument. The two are still comparable, because the uniform death rule matches the random death of the Moran process and birth is proportional to fitness in both. For a large population, Traulsen and Hauert show that these dynamics reduce to the replicator equation, which is consistent with the structure the uniform rule recovers.

Under the inverse death rule low-fitness individuals are more likely to die, so selection acts through death as well as through birth. This changes the process from the Moran case, where death is random, and the discovery reflects the change as the structure $\{p, p^3, p^4\}$. The two selection channels also drive the population to the equilibrium faster. Under the inverse rule the trajectories settle within 150 generations, while under the uniform rule, where selection acts through birth alone, they converge more slowly, as a comparison of Figure 5.3 and Figure 5.5 shows.

The match to the replicator under the uniform rule is not exact. The trajectory error stays small but non-zero, the estimated ESS sits a little above 0.75 at every averaging level, and trajectories that start above the ESS converge more slowly than the replicator predicts (Figure 5.5).

One possible cause of the slower descent, among several, is the way the min-max rescaling (Eq. (3.14)) interacts with the payoffs. Each individual holds a strategy, the

probability of playing Hawk, and earns payoffs from its encounters. Because the injury cost exceeds the resource ($c > v$), an encounter in which both individuals play Hawk can produce a negative payoff, and as these payoffs accumulate over the encounters of one interaction phase the least lucky individuals reach a very low fitness. The rescaling then measures reproduction against the lowest fitness in the population. Above the ESS the mean Hawk strategy is high and such encounters are frequent, so the minimum is dragged far below the rest of the population, the fitness differences between more and less aggressive individuals are compressed, and selection weakens, which slows the descent. Below the ESS the mean Hawk strategy is lower, fewer encounters have both individuals playing Hawk, and the accumulated payoffs are less negative, so the minimum stays close to the rest of the population and the fitness differences are not compressed as much. This could explain why the convergence speed is asymmetric above and below the ESS. Under the inverse death rule the delay probably does not arise, since the second selection channel removes low-fitness individuals through death as well.

6.2. Limitations and Difficulties

The discovery pipeline has two main limitations. The first is the candidate library, which is restricted to monomials up to degree five and contains no constant term. This span covers the replicator dynamics and a range of nearby polynomial models, but a governing equation built from other functional forms would lie outside it and could not be discovered.

The second is the trajectory cutoff T_{cut} , which involves a trade-off. The equilibrium region carries almost no dynamical information, since $\dot{p} \approx 0$, while the transient region carries the signal. Choosing the cutoff too early discards useful data, and choosing it too late adds noise to the regression, so the cutoff directly affects the result.

The genetic algorithm that generates the data brings three further limitations. The first concerns the initialization of the population. The population starts as a mixture of pure Hawks and pure Doves, chosen so that the mean Hawk strategy matches the target p_0 , and mixed strategies enter only later through mutation. This choice is not arbitrary. Other initializations were tested, including uniform and beta distributions over $[0, 1]$, but these produced crossing trajectories. Two populations with the same mean Hawk strategy $\bar{p}$ but different underlying strategy distributions then evolve differently, so $\bar{p}$ is no longer a sufficient state variable and no scalar autonomous equation $\dot{\bar{p}} = f(\bar{p})$ exists. The discovery here therefore only uses the pure-strategy initialization, which appeared to admit such a scalar description in the cases tested.

The second concerns the population size, which was increased from $N = 100$ in the reference paper to $N = 1,000$ because the trajectories at $N = 100$ were too noisy for reliable discovery. The sensitivity analysis over n_{avg} shows, however, that ensemble averaging already suppresses the stochastic noise, with the results essentially flat for $n_{\text{avg}} \geq 5$.

Increasing the population size and averaging over runs are two routes to the same noise reduction, so the larger population was probably not necessary. The flat part of the sensitivity analysis is still reported, as it documents the consistency of the results.

The third is that the coefficients of the models discovered for the genetic-algorithm datasets cannot be compared to a baseline equation, although v and c were chosen the same as in the synthetic data experiment. The reason is that the effective timescale of the trajectories is set mainly by N_c , the number of replacements per generation, which scales the coefficients by an unknown factor, so only the structure and the fixed points remain quantitatively interpretable.

6.3. Future Research

The main contribution of this work is the reusable discovery pipeline. Its real value lies in extracting macroscopic laws from simulations where no analytical equation exists, as already shown by the unexpected equation found for the inverse death rule. The natural next step is to apply it to higher-dimensional problems.

A natural two-dimensional extension is already contained in the reference paper. Besides the homogeneous game used here, Laruelle et al. [3] also study a heterogeneous version, in which the population is split into two types and each individual carries two strategies, the probability of playing Hawk against an opponent of each type. The macroscopic state is then the pair of mean strategies and is therefore two-dimensional, and the population no longer settles at a single ESS but oscillates around a neutrally stable set. Applying the discovery pipeline to this case would test it on a two-dimensional system whose dynamics converge to a set rather than a point.

A further extension would be to add a third action to the game. So far each individual plays either Hawk or Dove, but a third strategy could be included. The first step would be to check whether the genetic algorithm can be extended to three actions and still produce trajectories that can be simulated and averaged. If this works, the discovery pipeline could then be applied to recover the governing equations. With three actions the population state is no longer a single frequency but a pair of them, so the recovered dynamics would be two-dimensional, which also would test the pipeline beyond the one-dimensional case and support its generality.

If the two-dimensional case works, the same idea extends to higher dimensions. Many fields use genetic algorithms or agent-based simulations whose emergent quantities have no known governing equation, such as ecology, epidemiology, and economics, and the pipeline could be applied there to recover one.

7. CONCLUSION

The main objective of this work was to create a robust data-driven pipeline that recovers interpretable macroscopic governing equations directly from the trajectories of a genetic algorithm that simulates an evolutionary game. The Hawk-Dove game served as a controlled test case. Its macroscopic dynamics can be described by the replicator equation, which provided a known benchmark to validate the pipeline on synthetic data. The same game can also be simulated by a genetic algorithm, which produces trajectories that do not come from that equation and whose governing equation could therefore be really discovered. The three research questions are answered as follows.

The first question asked how a data-driven pipeline must be designed to reliably extract dynamical equations from noisy trajectories. The pipeline developed here combines Savitzky-Golay denoising, sparse regression with STLSQ, and AIC_c -based model selection. On the synthetic data it recovers the replicator equation with the correct structure and small coefficient errors up to a noise level of $\eta \leq 14\%$.

The second question asked what equation best describes the macroscopic dynamics of the genetic algorithm of Laruelle et al. [3] with the inverse death rule. The discovery process selects the structure $\{p, p^3, p^4\}$ rather than the replicator equation. Although the structure differs, the identified model reproduces the same fixed points, an ESS close to 0.75 and an unstable boundary point near 1, and therefore captures the same qualitative dynamics but by another equation.

The third question asked whether the discovery process detects a change in the microscopic death rule. It does. Changing the death rule from inverse-proportional to uniform changes the discovered structure from $\{p, p^3, p^4\}$ to the replicator structure $\{p, p^2, p^3\}$. The method therefore detects a microscopic change at the macroscopic level.

Across all three datasets the fixed points and the ESS are recovered, even where the full equation or its coefficients are not. This shows that interpretable macroscopic equations can be discovered from the stochastic output of a genetic algorithm, and that the discovery is sensitive to the microscopic rules that generate the data. The pipeline is general and could be applied to higher-dimensional simulations where no closed-form macroscopic equation is known.

BIBLIOGRAPHY

- [1] S. L. Brunton, J. L. Proctor, and J. N. Kutz, “Discovering governing equations from data by sparse identification of nonlinear dynamical systems,” *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [2] J. Maynard Smith and G. R. Price, “The logic of animal conflict,” *Nature*, vol. 246, no. 5427, pp. 15–18, 1973.
- [3] A. Laruelle, A. B. da Silva Rocha, and R. Escobedo, “The Hawk–Dove game in phenotypically homogeneous and heterogeneous populations of finite dimension,” *Communications in Nonlinear Science and Numerical Simulation*, vol. 55, pp. 56–67, 2018.
- [4] J. Bongard and H. Lipson, “Automated reverse engineering of nonlinear dynamical systems,” *Proceedings of the National Academy of Sciences*, vol. 104, no. 24, pp. 9943–9948, 2007.
- [5] M. Schmidt and H. Lipson, “Distilling free-form natural laws from experimental data,” *Science*, vol. 324, no. 5923, pp. 81–85, 2009.
- [6] H. Schaeffer, “Learning partial differential equations via data discovery and sparse optimization,” *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 473, no. 2197, 2017.
- [7] K. Champion, P. Zheng, A. Y. Aravkin, S. L. Brunton, and J. N. Kutz, “A unified sparse optimization framework to learn parsimonious physics-informed models from data,” *IEEE Access*, vol. 8, pp. 169 259–169 271, 2020.
- [8] K. Kaheman, J. N. Kutz, and S. L. Brunton, “SINDy-PI: A robust algorithm for parallel implicit sparse identification of nonlinear dynamics,” *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 476, no. 2242, 2020.
- [9] D. A. Messenger and D. M. Bortz, “Weak SINDy: Galerkin-based data-driven model selection,” *Multiscale Modeling & Simulation*, vol. 19, no. 3, pp. 1474–1497, 2021.
- [10] U. Fasel, J. N. Kutz, B. W. Brunton, and S. L. Brunton, “Ensemble-SINDy: Robust sparse model discovery in the low-data, high-noise limit, with active learning and control,” *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 478, no. 2260, 2022.
- [11] N. M. Mangan, J. N. Kutz, S. L. Brunton, and J. L. Proctor, “Model selection for dynamical systems via sparse regression and information criteria,” *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 473, no. 2204, p. 20 170 009, Aug. 2017.

- [12] B. M. de Silva, K. Champion, M. Quade, J.-C. Loiseau, J. N. Kutz, and S. L. Brunton, “PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data,” *Journal of Open Source Software*, vol. 5, no. 49, p. 2104, 2020.
- [13] A. A. Kaptanoglu et al., “PySINDy: A comprehensive Python package for robust sparse system identification,” *Journal of Open Source Software*, vol. 7, no. 69, p. 3994, 2022.
- [14] S. H. Rudy, S. L. Brunton, J. L. Proctor, and J. N. Kutz, “Data-driven discovery of partial differential equations,” *Science advances*, vol. 3, no. 4, e1602614, 2017.
- [15] J.-C. Loiseau and S. L. Brunton, “Constrained sparse galerkin regression,” *Journal of Fluid Mechanics*, vol. 838, pp. 42–67, 2018.
- [16] N. M. Mangan, S. L. Brunton, J. L. Proctor, and J. N. Kutz, “Inferring biological networks by sparse identification of nonlinear dynamics,” *IEEE Transactions on Molecular, Biological, and Multi-Scale Communications*, vol. 2, no. 1, pp. 52–63, 2016.
- [17] J. Horrocks and C. T. Bauch, “Algorithmic discovery of dynamic models from infectious disease data,” *Scientific reports*, vol. 10, no. 1, p. 7061, 2020.
- [18] J. T. Nardini, R. E. Baker, M. J. Simpson, and K. B. Flores, “Learning differential equation models from stochastic agent-based model simulations,” *Journal of The Royal Society Interface*, vol. 18, no. 176, Mar. 2021.
- [19] D. A. Messenger and D. M. Bortz, “Learning mean-field equations from particle data using WSINDy,” *Physica D: Nonlinear Phenomena*, vol. 439, p. 133 406, Nov. 2022.
- [20] A. Savitzky and M. J. Golay, “Smoothing and differentiation of data by simplified least squares procedures,” *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.
- [21] M. Elad, *Sparse and redundant representations: from theory to applications in signal and image processing*. Springer Science & Business Media, 2010.
- [22] S. Foucart and H. Rauhut, *A Mathematical Introduction to Compressive Sensing*. Birkhäuser, 2013.
- [23] B. K. Natarajan, “Sparse approximate solutions to linear systems,” *SIAM Journal on Computing*, vol. 24, no. 2, pp. 227–234, 1995.
- [24] R. Tibshirani, “Regression shrinkage and selection via the lasso,” *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 58, no. 1, pp. 267–288, 1996.
- [25] T. Hastie, R. Tibshirani, J. Friedman, et al., *The Elements of Statistical Learning*. Springer, 2009.
- [26] C. D. Manning, *Introduction to Information Retrieval*. Syngress Publishing, 2008.

- [27] J. M. Stevens-Haas, Y. Bhangale, J. Nathan Kutz, and A. Aravkin, “Learning non-linear dynamics using Kalman smoothing,” *IEEE Access*, vol. 12, pp. 138 564–138 574, 2024.
- [28] S. H. Strogatz, *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. Chapman and Hall/CRC, 2024.
- [29] J. Maynard Smith, *Evolution and the Theory of Games*. Cambridge University Press, 1982.
- [30] M. A. Nowak, *Evolutionary Dynamics: Exploring the Equations of Life*. Harvard University Press, 2006.
- [31] J. F. Nash, “Equilibrium points in n -person games,” *Proceedings of the National Academy of Sciences*, vol. 36, no. 1, pp. 48–49, 1950.
- [32] P. D. Taylor and L. B. Jonker, “Evolutionary stable strategies and game dynamics,” *Mathematical Biosciences*, vol. 40, no. 1-2, pp. 145–156, 1978.
- [33] J. Hofbauer and K. Sigmund, *Evolutionary games and population dynamics*. Cambridge university press, 1998.
- [34] R. A. Fisher, *The Genetical Theory of Natural Selection*, 2nd ed. New York: Dover Publications, Inc., 1958.
- [35] S. Wright, “Evolution in Mendelian populations,” *Genetics*, vol. 16, no. 2, p. 97, 1931.
- [36] P. A. P. Moran, “Random processes in genetics,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 54, no. 1, pp. 60–71, 1958.
- [37] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.
- [38] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [39] A. Traulsen and C. Hauert, “Stochastic evolutionary game dynamics,” *Reviews of nonlinear dynamics and complexity*, vol. 2, pp. 25–61, 2009.

A. SAVITZKY-GOLAY FILTER

This appendix gives the full procedure of the Savitzky-Golay filter introduced in Section 2.1.2 and shows how the same filter is used both to estimate the time derivative and to smooth the state values. The filter fits a local polynomial in a sliding window. Its two parameters are the window length $W = 2w + 1$ and the polynomial degree d . It operates on each state component independently, so it is enough to describe it for a single scalar component $\tilde{x}_k(t)$.

At each time step t_i , the window

$$\{t_{i-w}, \dots, t_i, \dots, t_{i+w}\} \quad (\text{A.1})$$

is selected. The points in the window are indexed by a local coordinate s that counts samples from the center. The center point t_i corresponds to $s = 0$, and the point at offset s is t_{i+s} , with s running over the integers $\{-w, \dots, w\}$. A polynomial

$$q_k(s) = \sum_{j=0}^d a_{j,k} s^j, \quad s \in \{-w, \dots, 0, \dots, w\} \quad (\text{A.2})$$

is fitted to the measured values $\tilde{x}_k(t_{i-w}), \dots, \tilde{x}_k(t_{i+w})$ by minimizing

$$\min_{a_{0,k}, \dots, a_{d,k}} \sum_{s=-w}^w \left(\tilde{x}_k(t_{i+s}) - \sum_{j=0}^d a_{j,k} s^j \right)^2. \quad (\text{A.3})$$

Writing the polynomial in s rather than in absolute time means the same fit can be reused at every point along the trajectory.

The fitted polynomial provides both quantities at the center of the window. Its value at $s = 0$ is the smoothed state, and its slope at $s = 0$ is the time derivative,

$$x_k(t_i) = q_k(0) = a_{0,k}, \quad \dot{x}_k(t_i) = \frac{a_{1,k}}{\Delta t}, \quad (\text{A.4})$$

where $\Delta t = t_{i+1} - t_i$ is the time step between measurements. The coefficient $a_{1,k}$ is the slope in sample-index units, and dividing by Δt converts it to physical time units. Collecting across all components gives the smoothed state vector $\mathbf{x}(t_i)$ and its derivative $\dot{\mathbf{x}}(t_i)$.

Near the boundaries of the trajectory the centered window is not available, since fewer than w points exist on one side. There an asymmetric window is used, which keeps the evaluation point fixed and uses only the available data.

B. PSEUDOCODE FOR THE GENETIC ALGORITHM

This appendix gives the full procedure of the genetic algorithm used to generate the genetic-algorithm data, following Laruelle et al. [3]. Algorithm 4 produces one trajectory of the mean Hawk strategy $\bar{p}$ by alternating an interaction phase and a reproduction phase over G generations. The two death rules studied in this work, inverse-proportional and uniform, enter only at the death step. Therefore the same procedure generates both datasets, with all parameters and referenced equations defined in the main text.

Algorithm 4 Genetic algorithm for the Hawk-Dove game (one trajectory), following [3].

```

1: Input:  $N, N_d, N_e, N_c, \mu, w_0, G, \alpha$ 
2: Initialize:  $p_i^0 \leftarrow 1$  for  $\lfloor \alpha N \rfloor$  individuals ( $H$ ),  $p_i^0 \leftarrow 0$  for the rest ( $D$ );  $w_i^0 \leftarrow w_0 \forall i \in \{1, \dots, N\}$ 
3: for  $t = 0$  to  $G - 1$  do
4:   // Interaction phase
5:   Sample  $N_d$  active individuals uniformly at random
6:   for each active individual  $i$  do
7:     for  $k = 1$  to  $N_e$  do
8:       Sample opponent  $j \neq i$  uniformly at random
9:       Individual  $i$  plays  $H$  w.p.  $p_i^t$ ,  $D$  w.p.  $1 - p_i^t$ ; resolve payoff  $\delta w_{i,k}^t$  (Table 3.2)
10:    end for
11:     $w_i^{t+1} \leftarrow w_i^t + \sum_{k=1}^{N_e} \delta w_{i,k}^t$ 
12:  end for
13:  Rescale all  $w_i^{t+1}$  by Eq. (3.14)
14:  // Reproduction phase
15:  for  $r = 1$  to  $N_c$  do
16:    Sample individual  $i$  to die:  $P_{\text{inverse}}$  (Eq. (4.3)) or  $P_{\text{uniform}}$  (Eq. (4.4))
17:    Sample individual  $k \neq i$  to reproduce with  $P_{\text{birth}}(k)$  (Eq. (3.15))
18:     $p_i^{t+1} \leftarrow p_k^{t+1}, w_i^{t+1} \leftarrow w_k^{t+1}$ 
19:    if  $\mathcal{U}[0, 1] < \mu$  then
20:       $p_i^{t+1} \sim \mathcal{U}[0, 1], w_i^{t+1} \leftarrow w_0$ 
21:    end if
22:  end for
23:  Record  $\bar{p}^{t+1} = \frac{1}{N} \sum_{i=1}^N p_i^{t+1}$ 
24: end for
25: Return:  $\{\bar{p}^t\}_{t=0}^G$ 

```

C. MODEL EVALUATION

This appendix reports the full evaluation of the best identified model at every sweep level, the noise level η for the synthetic data and the number of averaged runs n_{avg} for the genetic-algorithm data, as referenced throughout Chapter 5. Each row lists the identified equation together with its evaluation metrics, the structure score F_1 , the coefficient error MAE_{Ξ} where the true coefficients are known, the interior fixed point $\hat{p}^*$ that estimates the ESS with its absolute error $\text{AE}_{\hat{p}^*}$, the boundary fixed point $\hat{r}_1$ with absolute error and any further real root, and the trajectory error $\overline{\text{RMSE}}$. The stability of each fixed point is marked S for stable and U for unstable.

C.1. Synthetic Data

Table C.1 reports the evaluation on the synthetic data across the eleven noise levels. The replicator structure $\{p, p^2, p^3\}$ is recovered up to $\eta = 14\%$, after which the identified structure changes.

TABLE C.1. SYNTHETIC DATA - BEST MODEL EVALUATION

η (%)	Identified equation ($\dot{p} =$)	F_1	MAE_{Ξ}	ESS estimate			Boundary fixed point			$\overline{\text{RMSE}}$
				$\hat{p}^*$	$\text{AE}_{\hat{p}^*}$	Stab.	$\hat{r}_1$	$\text{AE}_{\hat{r}_1}$	Stab.	
0	$0.150p - 0.351p^2 + 0.201p^3$	1.000	4.4×10^{-4}	0.749	0.001	S	0.999	0.001	U	4.1×10^{-4}
2	$0.149p - 0.345p^2 + 0.195p^3$	1.000	2.0×10^{-3}	0.756	0.006	S	1.011	0.011	U	6.7×10^{-3}
4	$0.149p - 0.340p^2 + 0.191p^3$	1.000	4.1×10^{-3}	0.764	0.014	S	1.020	0.020	U	1.3×10^{-2}
6	$0.148p - 0.336p^2 + 0.187p^3$	1.000	5.7×10^{-3}	0.771	0.021	S	1.025	0.025	U	1.9×10^{-2}
8	$0.148p - 0.334p^2 + 0.186p^3$	1.000	6.5×10^{-3}	0.778	0.028	S	1.024	0.024	U	2.5×10^{-2}
10	$0.148p - 0.334p^2 + 0.186p^3$	1.000	6.4×10^{-3}	0.786	0.036	S	1.015	0.015	U	3.2×10^{-2}
12	$0.149p - 0.336p^2 + 0.188p^3$	1.000	5.4×10^{-3}	0.795	0.045	S	0.998	0.002	U	3.9×10^{-2}
14	$0.150p - 0.341p^2 + 0.192p^3$	1.000	3.5×10^{-3}	0.806	0.056	S	0.971	0.029	U	4.7×10^{-2}
16	$0.242p - 0.616p^2$	0.571	0.466	0.895	0.145	S	-0.762	1.762	S	6.6×10^{-2}
	$+ 1.066p^4 - 0.709p^5$									
18	$0.252p - 0.648p^2$	0.571	0.496	0.910	0.160	S	-0.759	1.759	S	7.5×10^{-2}
	$+ 1.130p^4 - 0.749p^5$									
20	$0.258p - 0.665p^2$	0.571	0.508	0.927	0.177	S	-0.761	1.761	S	8.3×10^{-2}
	$+ 1.155p^4 - 0.760p^5$									

C.2. GA Data (Inverse Death)

Table C.2 reports the evaluation on the genetic-algorithm data under the inverse death rule across the averaging levels. The identified structure $\{p, p^3, p^4\}$ carries a third root $\hat{r}_2$, which lies outside $[0, 1]$ and does not affect the dynamics.

TABLE C.2. GA DATA (INVERSE DEATH) - BEST MODEL
EVALUATION

n_{avg}	Identified equation ($\dot{p} =$)	F_1	ESS estimate				Other fixed points					$\overline{\text{RMSE}}$
			$\hat{p}^*$	$\text{AE}_{\hat{p}^*}$	Stab.		$\hat{r}_1$	$\text{AE}_{\hat{r}_1}$	Stab.	$\hat{r}_2$	Stab.	
1	$0.081p - 0.178p^2 + 0.094p^3$	1.000	0.769	0.019	S		1.115	0.115	U	—	—	2.5×10^{-2}
5	$0.048p - 0.202p^3 + 0.155p^4$	0.667	0.753	0.003	S		0.977	0.023	U	-0.425	S	1.2×10^{-2}
10	$0.048p - 0.199p^3 + 0.152p^4$	0.667	0.753	0.003	S		0.979	0.021	U	-0.426	S	9.5×10^{-3}
15	$0.047p - 0.196p^3 + 0.150p^4$	0.667	0.755	0.005	S		0.982	0.018	U	-0.427	S	9.2×10^{-3}
20	$0.048p - 0.197p^3 + 0.150p^4$	0.667	0.754	0.004	S		0.983	0.017	U	-0.427	S	8.5×10^{-3}
25	$0.047p - 0.196p^3 + 0.149p^4$	0.667	0.754	0.004	S		0.987	0.014	U	-0.428	S	8.8×10^{-3}
30	$0.048p - 0.196p^3 + 0.149p^4$	0.667	0.755	0.005	S		0.986	0.014	U	-0.428	S	8.2×10^{-3}
35	$0.047p - 0.196p^3 + 0.149p^4$	0.667	0.755	0.005	S		0.987	0.013	U	-0.428	S	8.4×10^{-3}
40	$0.047p - 0.196p^3 + 0.149p^4$	0.667	0.754	0.004	S		0.986	0.014	U	-0.428	S	8.5×10^{-3}

C.3. GA Data (Uniform Death)

Table C.3 reports the evaluation on the genetic-algorithm data under the uniform death rule across the averaging levels, where the replicator structure $\{p, p^2, p^3\}$ is recovered at every level.

TABLE C.3. GA DATA (UNIFORM DEATH) - BEST MODEL
EVALUATION

n_{avg}	Identified equation ($\dot{p} =$)	F_1	ESS estimate				Boundary fixed point			$\overline{\text{RMSE}}$
			$\hat{p}^*$	$\text{AE}_{\hat{p}^*}$	Stab.		$\hat{r}_1$	$\text{AE}_{\hat{r}_1}$	Stab.	
1	$0.062p - 0.139p^2 + 0.077p^3$	1.000	0.782	0.032	S		1.029	0.029	U	3.6×10^{-2}
5	$0.059p - 0.132p^2 + 0.073p^3$	1.000	0.786	0.036	S		1.034	0.034	U	2.0×10^{-2}
10	$0.057p - 0.127p^2 + 0.068p^3$	1.000	0.779	0.029	S		1.074	0.074	U	1.8×10^{-2}
15	$0.057p - 0.127p^2 + 0.068p^3$	1.000	0.777	0.027	S		1.079	0.079	U	1.9×10^{-2}
20	$0.058p - 0.128p^2 + 0.069p^3$	1.000	0.776	0.026	S		1.080	0.080	U	2.0×10^{-2}
25	$0.058p - 0.129p^2 + 0.070p^3$	1.000	0.776	0.026	S		1.077	0.077	U	2.0×10^{-2}
30	$0.058p - 0.129p^2 + 0.070p^3$	1.000	0.776	0.026	S		1.074	0.074	U	2.0×10^{-2}
35	$0.059p - 0.131p^2 + 0.071p^3$	1.000	0.776	0.026	S		1.071	0.071	U	1.9×10^{-2}
40	$0.059p - 0.130p^2 + 0.070p^3$	1.000	0.775	0.025	S		1.077	0.077	U	1.9×10^{-2}